WIA1006/WID3006
MACHINE LEARNING

Semester 2, Session 2025/2026 | Universiti Malaya

EXPECTATION MAXIMIZATION (EM)

Recap

Gaussian Mixture Model (GMM) creates a new pdf for us to generate random variables, with the following distribution.

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$$

with π_k the mixing coefficients, where

$$\sum_{k=1}^{K} \pi_k = 1 \quad \text{and} \quad \pi_k \geq 0 \quad \forall k$$

- GMM is a density estimator
- GMMs are universal approximators of density (if you have enough Gaussians).

Recap

Maximum Likelihood Estimation of a GMM

Mean $$\mu_k = \frac{\sum_{n=1}^{N} \tau(z_{nk}) x_n}{N_k}$$

Covariance $$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^{N} \tau(z_{nk})(x_n - \mu_k)(x_n - \mu_k)^T$$

Mixing coefficient $$\pi_k = \frac{N_k}{N}$$

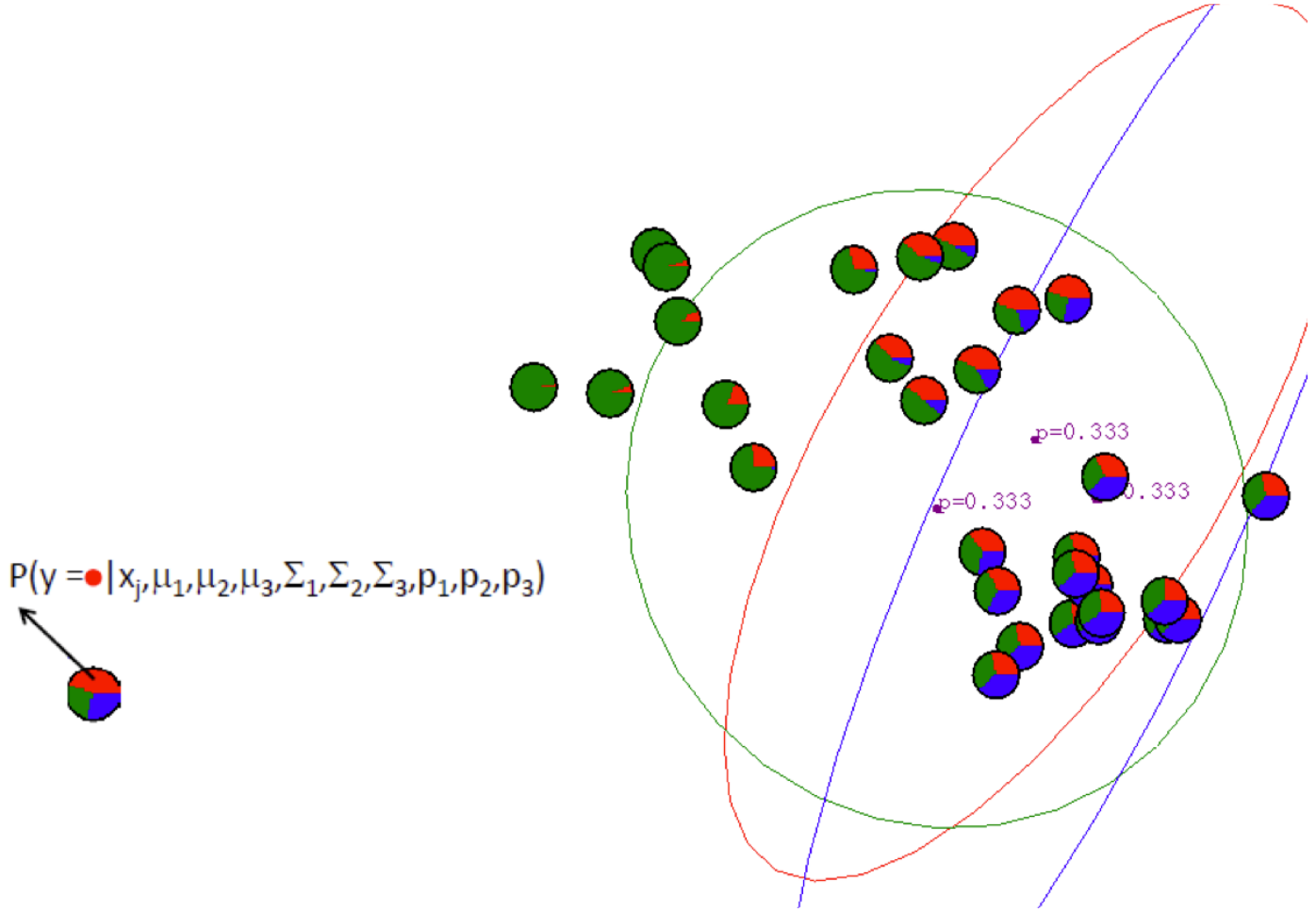
$$N_k = \sum_{n=1}^{N} \tau(z_{nk})$$

Not a closed form solution!!
Use Expectation-Maximization Algorithm

EM Algorithm

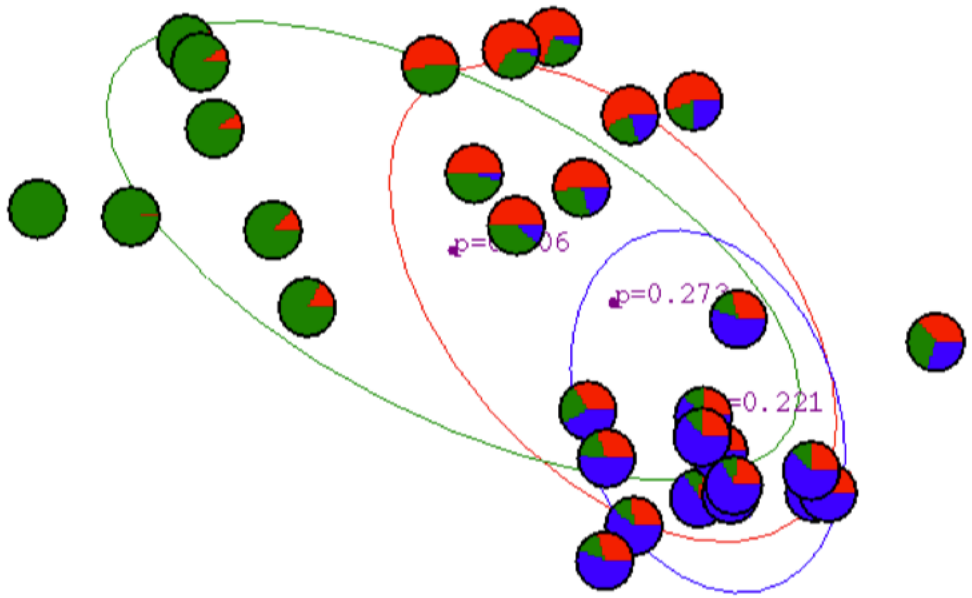
- Expectation Maximization (EM) is a general algorithm to deal with hidden variables.
- Two steps:
  - E Step: Compute the posterior probability over z given our current model - i.e. how much do we think each Gaussian generates each datapoint.
  - M Step: Assuming that the data really was generated this way, change the parameters of each Gaussian to maximize the probability that it would generate the data it is currently responsible for. In another word, maximize the probability that it would generate the data it is currently responsible for.

EM Algorithm for GMM



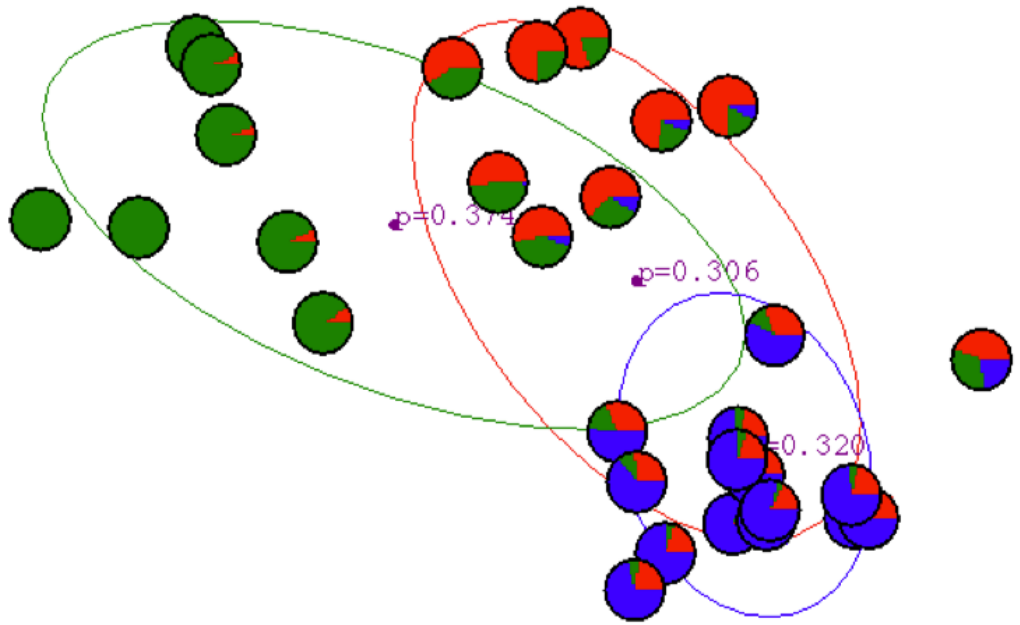
EM Algorithm for GMM

After 1st iteration



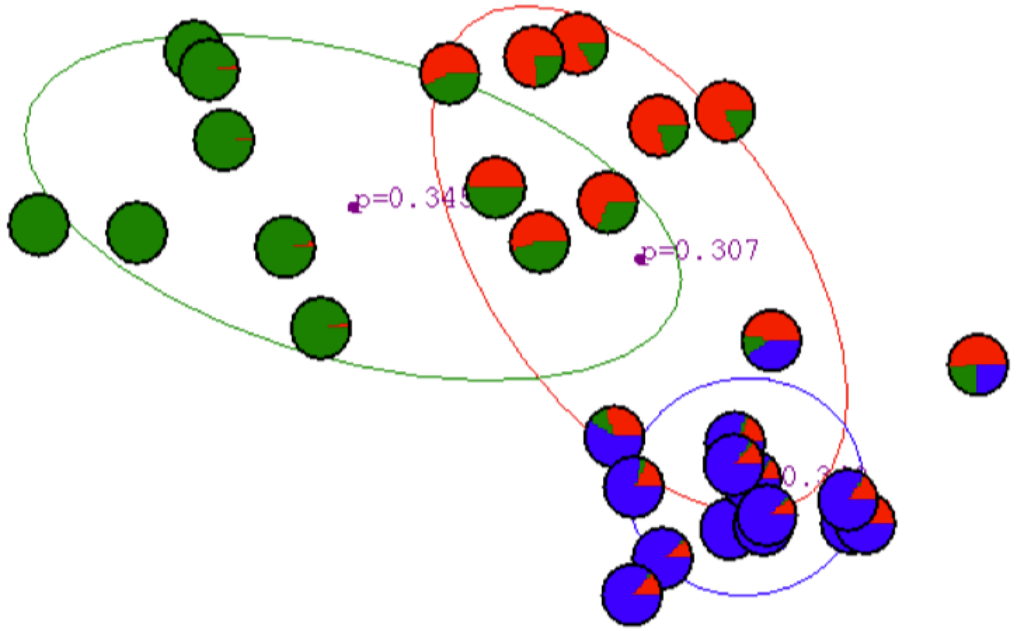
EM Algorithm for GMM

After 2nd iteration



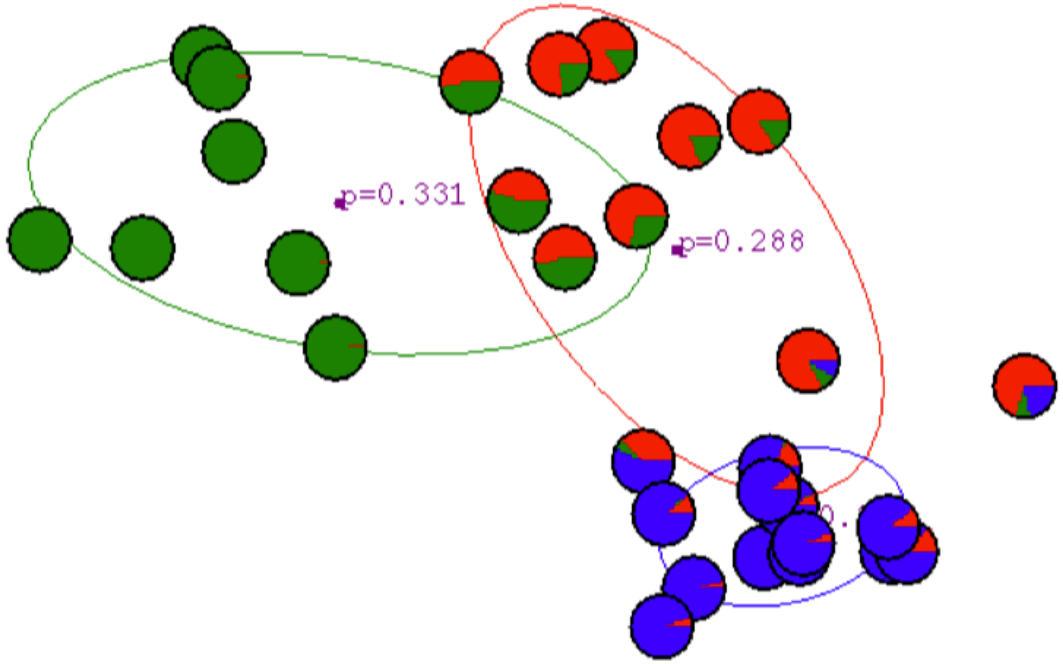
EM Algorithm for GMM

After 3rd iteration



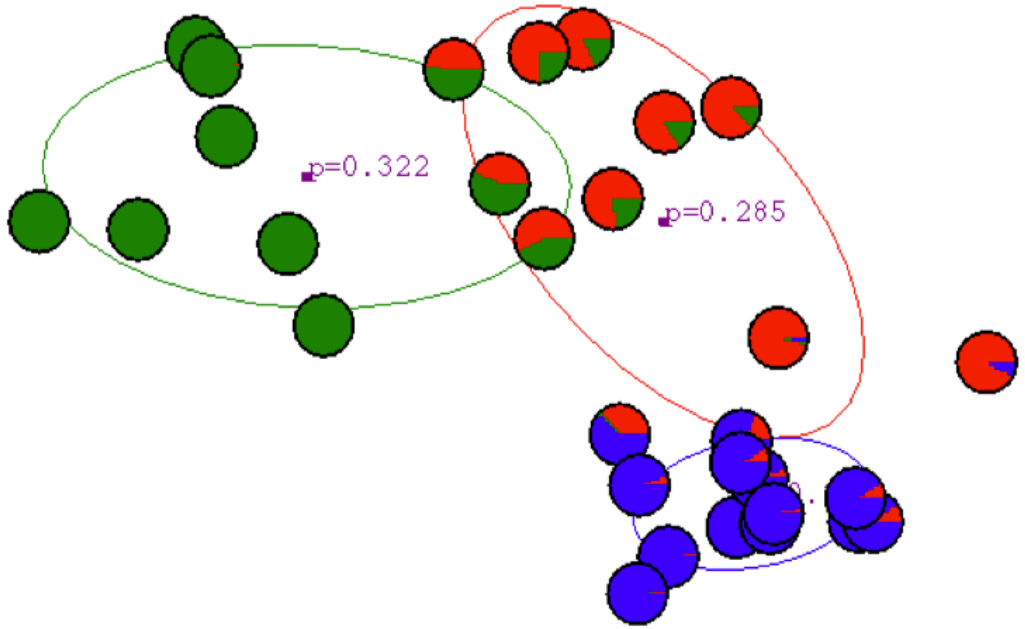
EM Algorithm for GMM

After 4th iteration



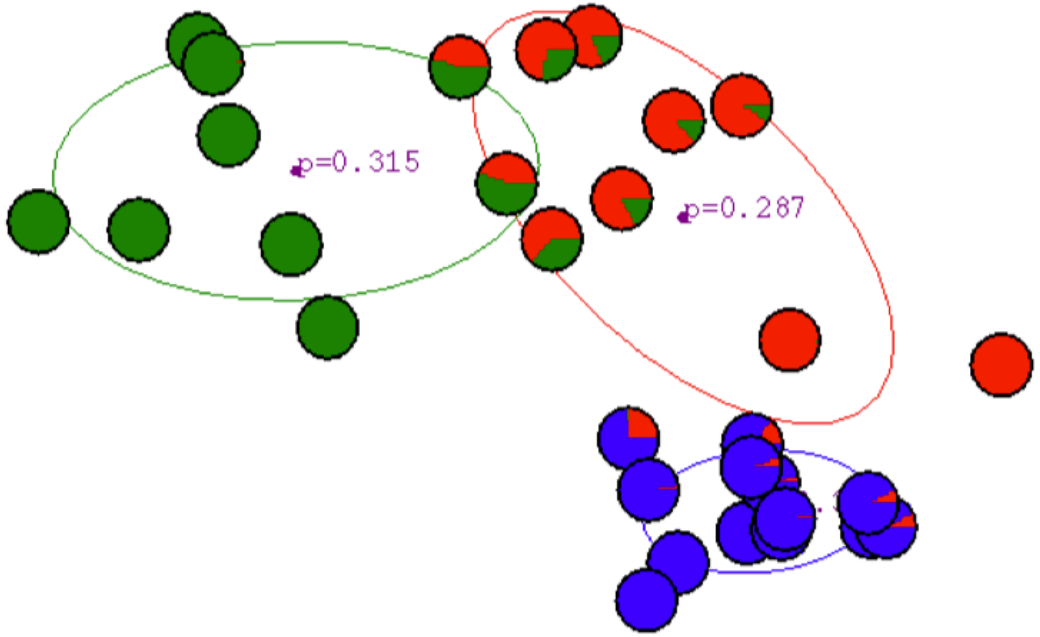
EM Algorithm for GMM

After 5th iteration



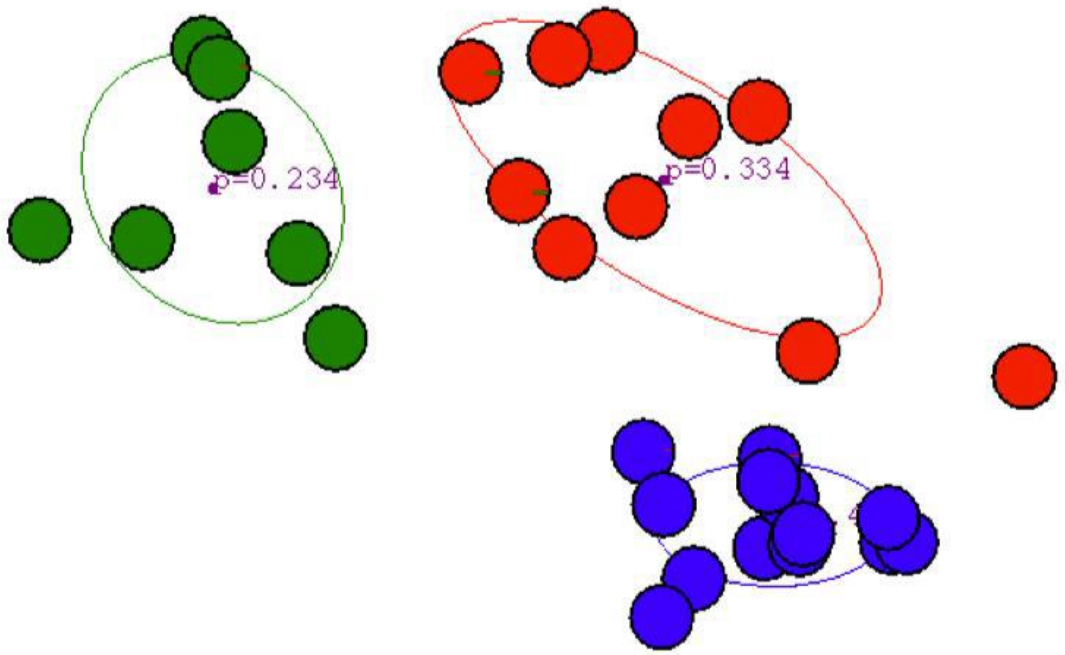
EM Algorithm for GMM

After 6th iteration



EM Algorithm for GMM

After 20th iteration



EM Algorithm for GMM

Given a GMM, the goal is to maximize the likelihood function with respect to the parameters comprising the means and covariances of the components and the mixing coefficients.

1. Initialize the mean, μ_i, covariances Σ_i, and mixing coefficients, π_i, and evaluate the initial value of the log likelihood.
2. E step. Evaluate the responsibilities using the current parameter values.

$$\tau(z_{nk}) = \frac{\pi_k N(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j N(x_n|\mu_j, \Sigma_j)}$$

EM Algorithm for GMM

3. M step: Re-estimate Parameters

$$\mu_k^{new} = \frac{\sum_{n=1}^{N} \tau(z_{nk}) x_n}{N_k}$$

$$\Sigma_k^{new} = \frac{1}{N_k} \sum_{n=1}^{N} \tau(z_{nk})(x_n - \mu_k^{new})(x_n - \mu_k^{new})^T$$

$$\pi_k^{new} = \frac{N_k}{N}$$

EM Algorithm for GMM

4. Evaluate the log likelihood

$$\ln p(X \mid \mu, \Sigma, \pi) = \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k N(x_n \mid \mu_k, \Sigma_k) \right\}$$

If there is no convergence, return to step 2.

Relationship to K-means

- K-means makes **hard** decisions.
  - Each data point gets assigned to a single cluster.
- GMM/EM makes **soft** decisions.
  - Each data point can yield a posterior $p(z|x)$
- K-means is a special case of EM.

General form of EM Algorithm

- Given a joint distribution over observed and latent variables: $p(X, Z|\theta)$
- Want to maximize: $p(X|\theta)$

1. Initialize parameters: θ^{old}
2. E Step: Evaluate: $p(Z|X, \theta^{old})$
3. M-Step: Re-estimate parameters (based on expectation of complete data log likelihood)

$$\theta^{new} = argmax_{\theta} \sum_{Z} p\left(Z\middle|X, \theta^{old}\right) \ln p(X, Z|\theta)$$

4. Check for convergence of params or likelihood

General form of EM Algorithm

Using Jensen's inequality

$$\ell(\theta; x) = \log p(x \mid \theta)$$

$$= \log \sum_z p(x, z \mid \theta)$$

$$= \log \sum_z q(z \mid x) \frac{p(x, z \mid \theta)}{q(z \mid x)}$$

$$\geq \sum_z q(z \mid x) \log \frac{p(x, z \mid \theta)}{q(z \mid x)}$$

Will lead to maximizing this

Maximizing this

Jensen's Inequality

$$
\begin{aligned}
F(q,\theta) &= \sum_z q(z \mid x) \log \frac{p(x,z \mid \theta)}{q(z \mid x)} \\
&= \sum_z q(z \mid x) \log p(x,z \mid \theta) - \sum_z q(z \mid x) \log q(z \mid x) \\
&= \left\langle \ell_c(\theta; x, z) \right\rangle_q + H_q
\end{aligned}
$$

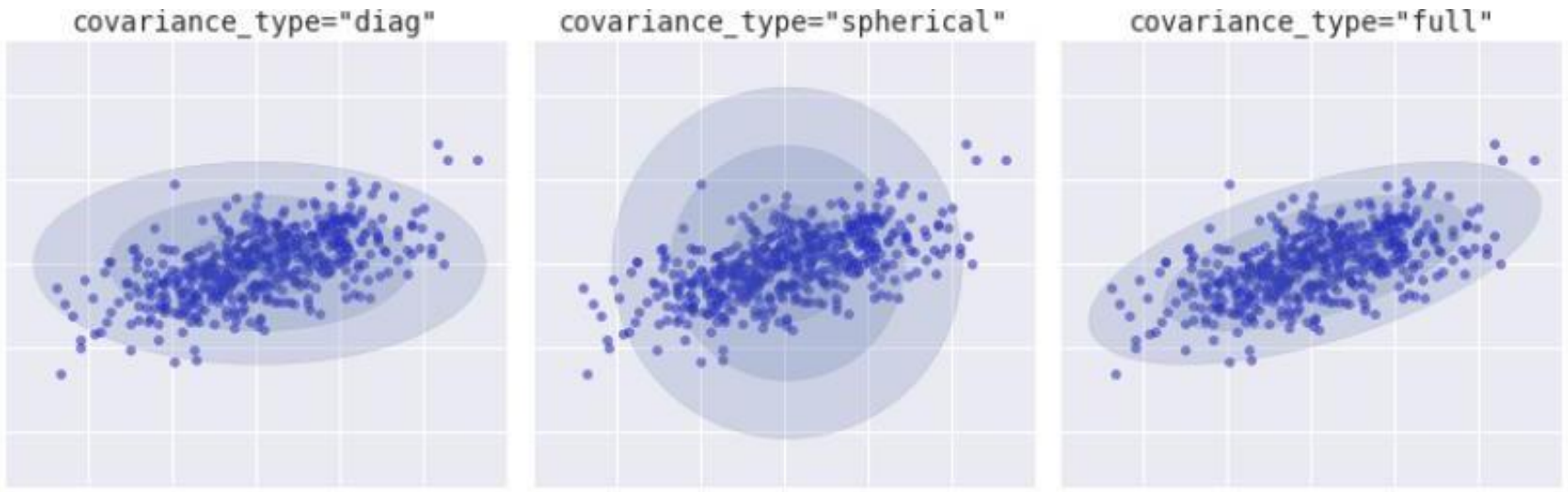
The first term is the expected complete log likelihood and the second term, which does not depend on θ, is the entropy.

Thus, in the M-step, maximizing with respect to θ for fixed q we only need to consider the first term:

$$
\theta^{t+1} = \arg\max_\theta \left\langle \ell_c(\theta; x, z) \right\rangle_{q^{t+1}} = \arg\max_\theta \sum q(z \mid x) \log p(x,z \mid \theta)
$$

EM Algorithm for GMM: Example

covariance_type="diag“ or "spherical“ or “full”



Source: Python Data Science Handbook by Jake VanderPlas

EM Algorithm Demo Link

https://lukapopijac.github.io/gaussian-mixture-model/

1. Try with creating different points on the demo link with different clusters.
2. Run 1 iteration each time and see how's the cluster change.

REINFORCEMENT LEARNING (RL)

Reinforcement Learning

Figure: An agent ...

Reinforcement Learning

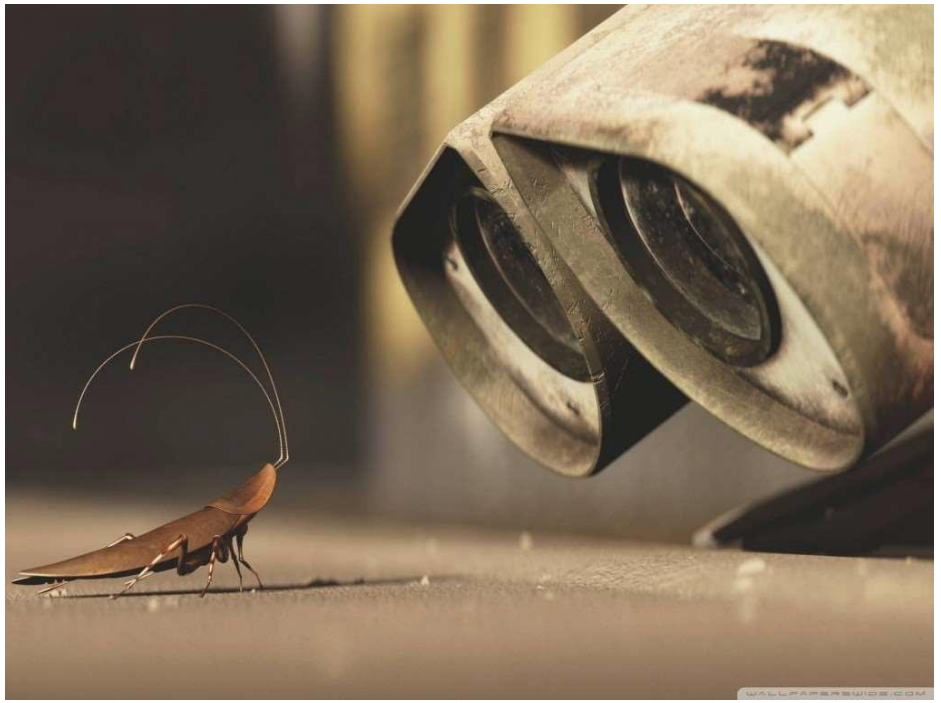


Figure: ... observes the world ...

Reinforcement Learning

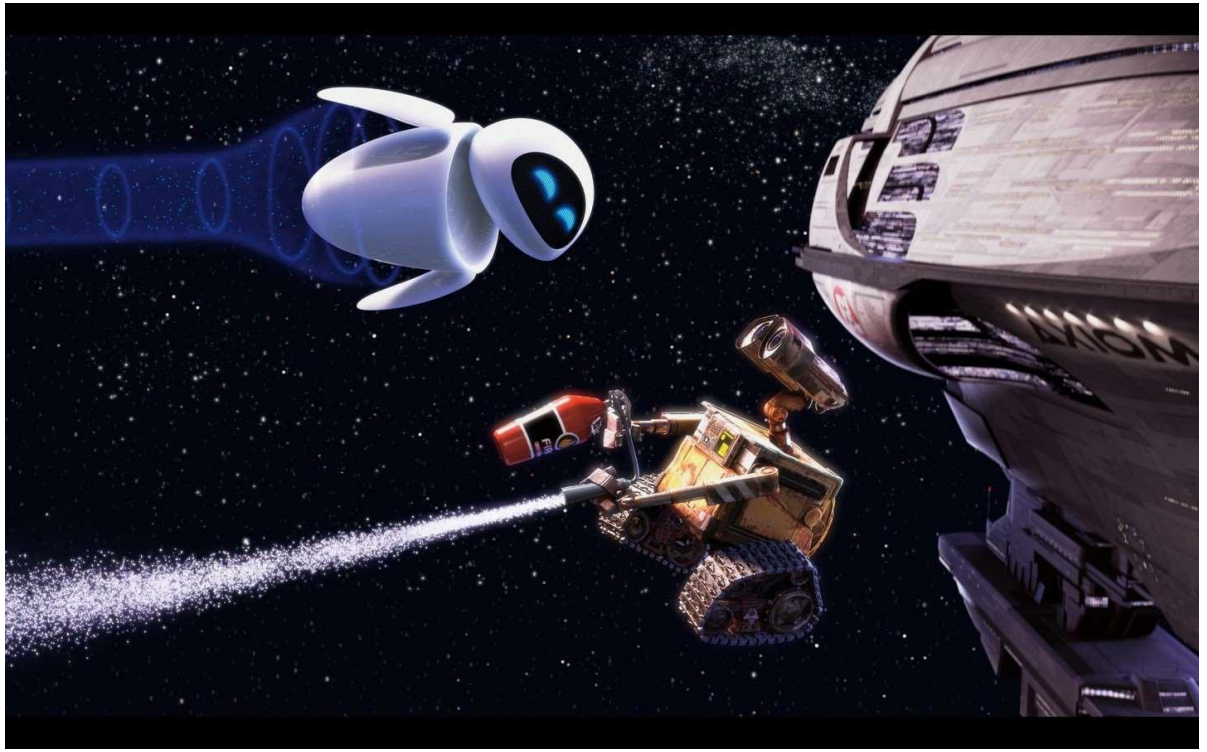


Figure: ... takes an action and its states changes ...

Reinforcement Learning

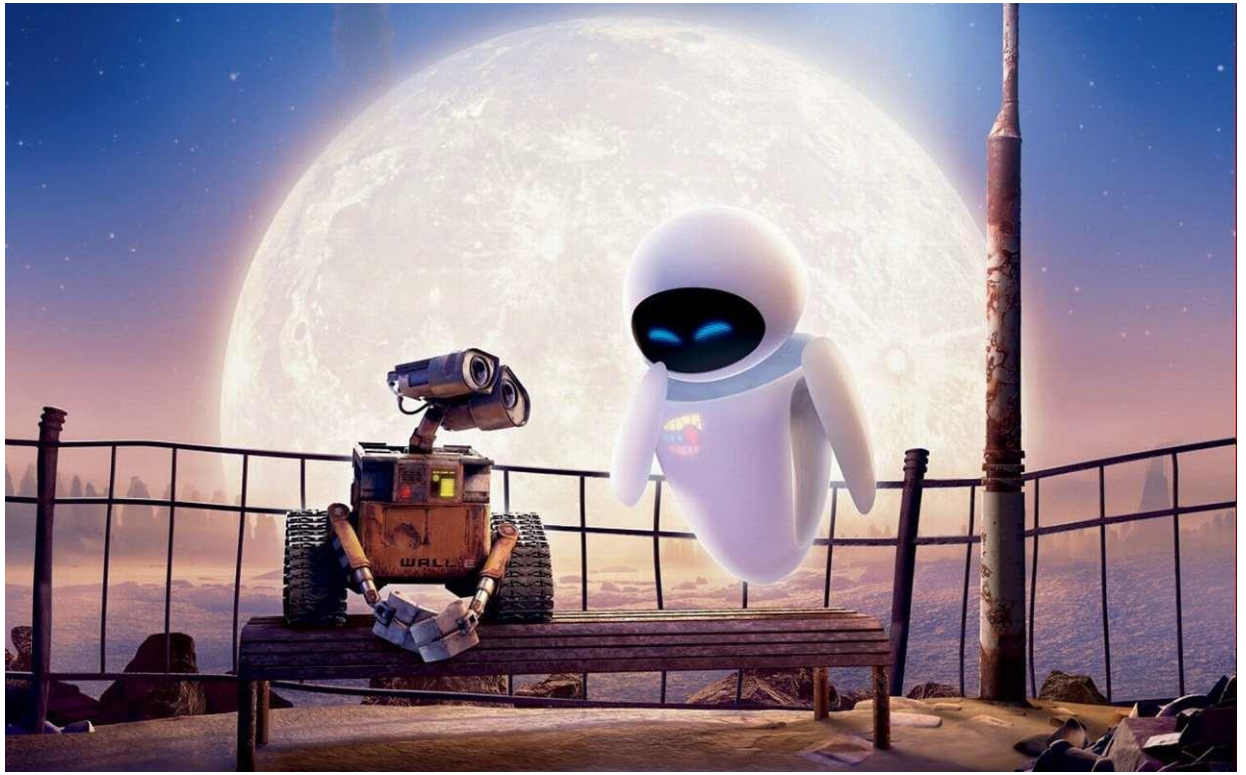


Figure: ... with the goal of achieving long-term rewards.

Reinforcement Learning in the News


LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

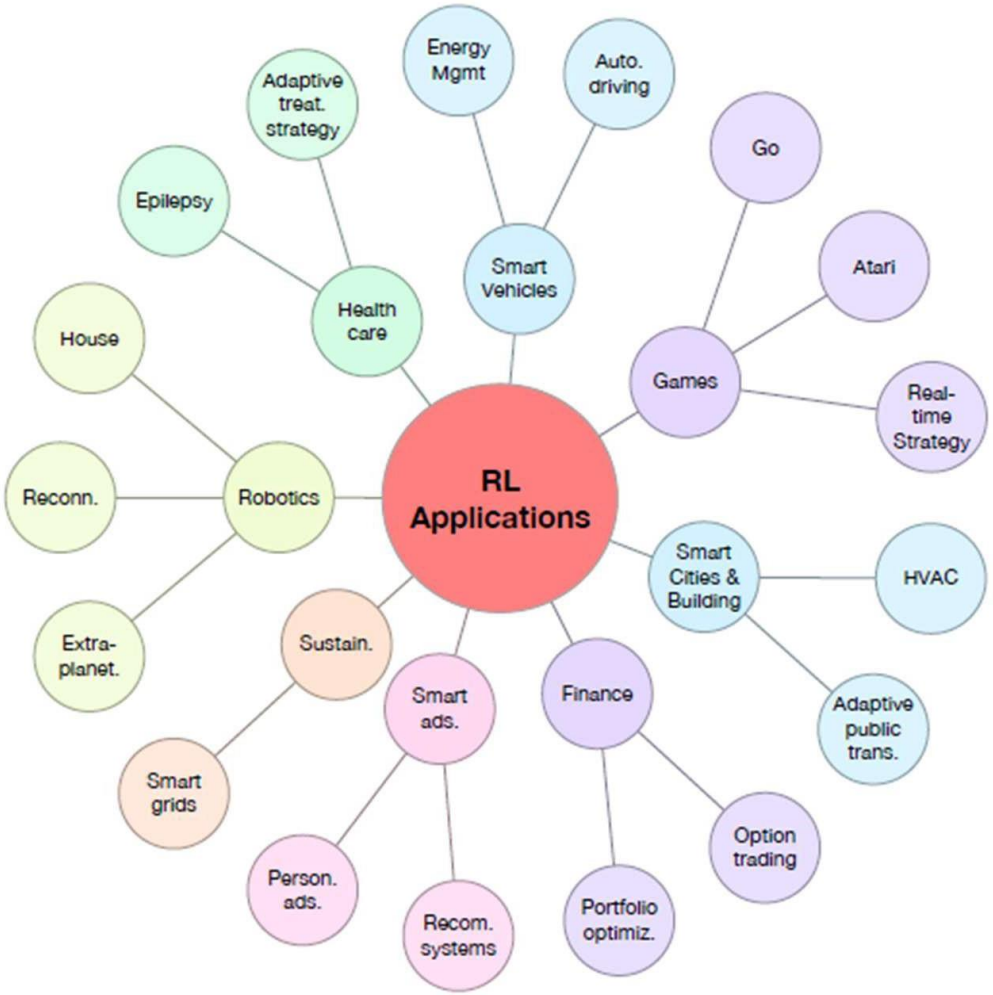

ARTICLE

doi:10.1038/nature16961

Mastering the game of Go with deep neural networks and tree search

David Silver[1]*, Aja Huang[1]*, Chris J. Maddison[1], Arthur Guez[1], Laurent Sifre[1], George van den Driessche[1], Julian Schrittwieser[1], Ioannis Antonoglou[1], Veda Panneershelvam[1], Marc Lanctot[1], Sander Dieleman[1], Dominik Grewe[1], John Nham[2], Nal Kalchbrenner[1], Ilya Sutskever[2], Timothy Lillicrap[1], Madeleine Leach[1], Koray Kavukcuoglu[1], Thore Graepel[1] & Demis Hassabis[1]

(Potential) Applications in RL



Reinforcement Learning

Reinforcement Learning (RL) refers to both a type of problem and a set of computational methods.

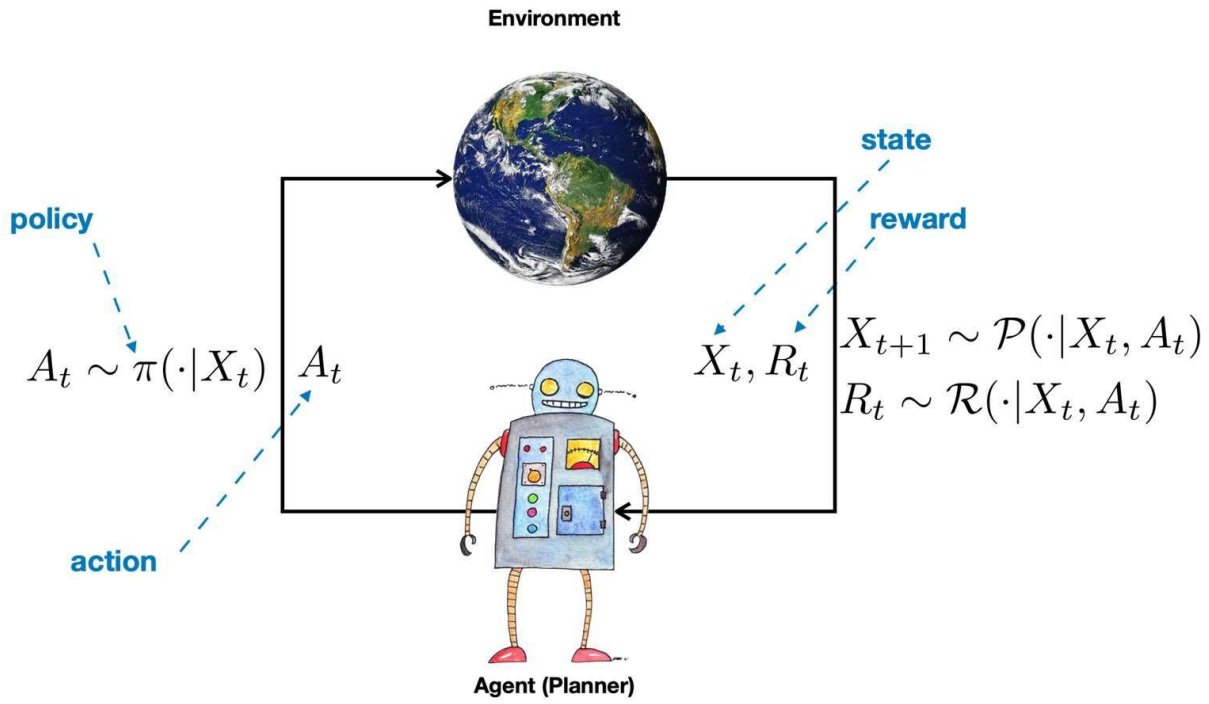
- **Problem:** How to act so that some notion of long-term performance is maximized?
- **Methods:** What kind of computation does an agent need to do in order to ensure that its actions lead to good (or even optimal) long-term performance?

Remark

Historically, only a subset of all computational methods that attempt to solve the RL problem are known as the RL methods, e.g., Q-Learning is, evolutionary computation methods are not.

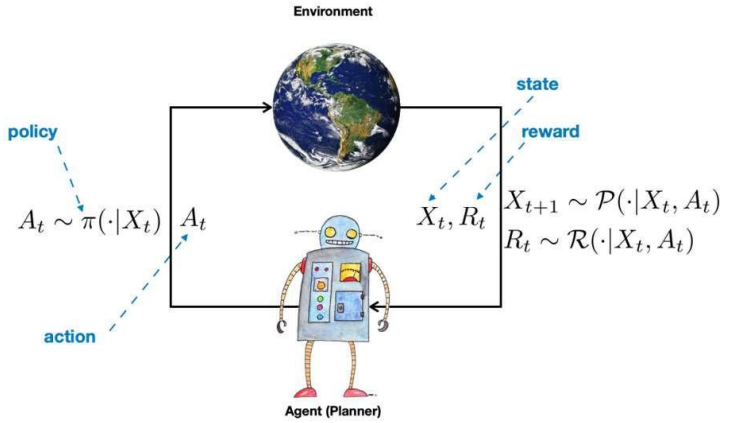
RL Problem and Agent-Environment Interaction

In RL, we often talk about an agent and its environment, and their interaction.



RL Problem and Agent-Environment Interaction

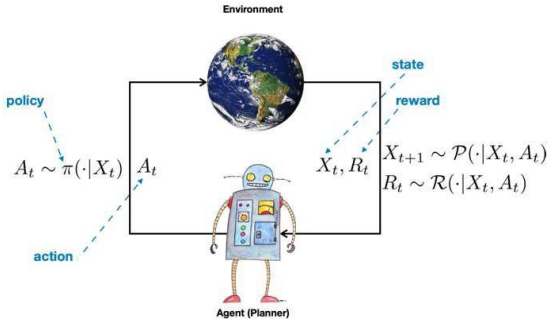
- Agent: decision maker and/or learner
  - robot
  - medical diagnosis and treatment system
  - air conditioning system
- Environment: anything outside the agent with which it interacts and attempts to control.
  - physical world outside the robot
  - patient's body
  - room



RL Problem and Agent-Environment Interaction

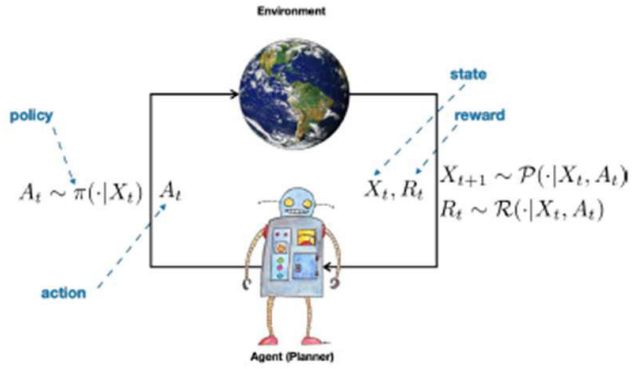
At time $t = 1, 2, \ldots,$ the interaction of the agent and the environment is as follows:

- the agent observes its state X_t in the environment.
  - Examples: position of the robot, vital information of a patient, the room temperature, etc.
- The agent picks an action A_t according to its policy π, e.g., $A_t = \pi(X_t)$ or $A_t \sim \pi(\cdot|X_t)$.
- The state of the agent in the environment changes and becomes X_{t+1} according to transition probability kernel (or distribution), i.e., $X_{t+1} \sim \mathcal{P}(\cdot|X_t, A_t)$.
- The agent also receives a reward signal R_t, i.e., $R_t \sim \mathcal{R}(\cdot|X_t, A_t)$.



RL Problem and Agent-Environment Interaction

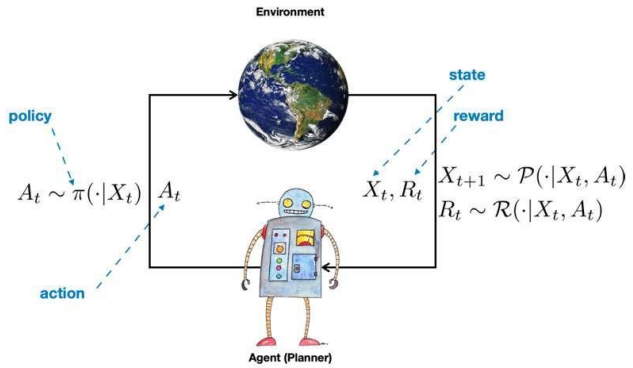
State: A variable that summarizes whatever has happened to the agent so far.
Policy: Action selection mechanism. Usually a mapping from states to actions. It can be deterministic ($A_t = \pi(X_t)$) or stochastic ($A_t \sim \pi(\cdot|X_t)$).
Transition probability kernel: Describes the dynamics. For example, a set of electromechanical equations describing how the position of the robot (including its joints) change when a certain command is sent to its motor. Or how the patient's physiology changes after the administration of the treatment.



RL Problem and Agent-Environment Interaction

Reward: A real number specifying the *immediate* desirability of the choice of action A_t at the state X_t (possibly leading to state X_{t+1}) has been. Examples:

- Positive if robot successfully picks up an object, negative if it breaks the object.
- Infection subsides
- The room temperature becomes comfortable.



Remark

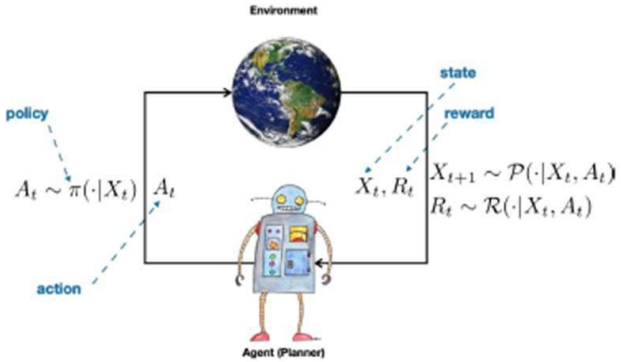
The reward signal/function/distribution only encodes the desirability of the action from the immediate perspective. A good action now may not be good in the long-term.

RL Problem and Agent-Environment Interaction

This process repeats and as a result, the agent receives a sequence of state, actions, and rewards:

$$X_1, A_1, R_1, X_2, A_2, R_2, \cdots .$$

This sequence might terminate after a fixed number of time steps (say, T), or until the agent gets to a certain region of the state space, or it might continue forever.



Markov Decision Process (MDP)

Let us formally define some important concepts that we require throughout the course. Beforehand, some commonly used notations:

Given a space Ω.

- $\mathcal{M}(\Omega)$: the space of all probability distributions defined over the space Ω.
- $B(\Omega)$: the space of all bounded functions defined over Ω

Examples: $\Omega = \{1, 2, \ldots, n\}, \mathbb{N}, \mathbb{R}, \mathbb{R}^d$, etc.

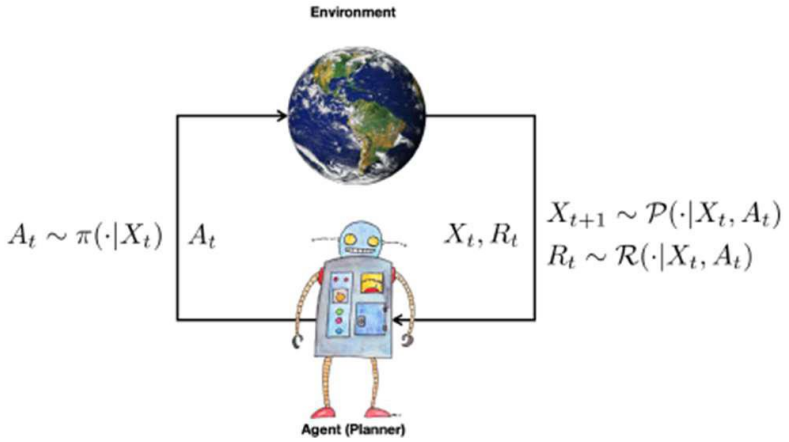
$\mathcal{M}(\mathbb{R})$: The space of distributions on the real line

$B(\mathbb{R})$: The space of bounded functions on the real line

Markov Decision Process (MDP)

Definition

A *discounted MDP* is a 5-tuple $(\mathcal{X}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{X}$ is a measurable state space, $\mathcal{A}$ is the action space,
$\mathcal{P} : \mathcal{X} \times \mathcal{A} \to \mathcal{M}(\mathcal{X})$ is the transition probability kernel with domain $\mathcal{X} \times \mathcal{A}$,
$\mathcal{R} : \mathcal{X} \times \mathcal{A} \to \mathcal{M}(\mathbb{R})$ is the immediate reward distribution, and $0 \leq \gamma < 1$ is the discount factor.



Markov Decision Process (MDP)

MDPs encode the temporal evolution of a discrete-time stochastic process controlled by an *agent*.

- Initial state $X_1 \sim \rho$ with $\rho \in \mathcal{M}(\mathcal{X})$.
- Agent chooses action $A_t \in \mathcal{A}$.
- Agent goes to $X_{t+1} \sim \mathcal{P}(\cdot|X_t, A_t)$ and receives reward $R_t \sim \mathcal{R}(\cdot|X_t, A_t)$.
- The process repeats. The trajectory is $\xi = (X_1, A_1, R_1, X_2, A_2, R_2, \cdots)$, which is random.

Remark

The reward distribution could also depend on the next-state X_{t+1}. In that case, we would have a different reward kernel $\mathcal{R}'$ and the reward would be $R_t \sim \mathcal{R}'(\cdot|X_t, A_t, X_{t+1})$. But we can absorb the dynamics within $\mathcal{R}$, i.e., $\mathcal{R}(\cdot|x, a) = \int \mathcal{R}'(\cdot|x, a, x')\mathcal{P}(\mathrm{d}x'|x, a)$.

Markov Decision Process (MDP)

This is a general framework.

State space:

- Finite: $\mathcal{X} = \{x_1, x_2, \ldots, x_n\}$ (or $\mathcal{X} = \{1, 2, \ldots, n\}$) with $n < \infty$
- Infinite but countable: $\mathcal{X} = \{x_1, x_2, \ldots\}$ (or $\mathcal{X} = \mathbb{N}$)
- Continuous: $\mathcal{X} \subset \mathbb{R}^d$

Dynamics:

- Stochastic
- Deterministic

Remark

A deterministic dynamical system always behave exactly the same given the same starting state and action. They can be described by the transition function $f : \mathcal{X} \times \mathcal{A} \to \mathcal{X}$ *and* $x_{t+1} = f(x_t, a_t)$.

Policy

Definition

A policy is a sequence $\bar{\pi} = \{\pi_1, \pi_2, \ldots\}$ such that for each t,

$$\pi_t(a_t | X_1, A_1, X_2, A_2, \ldots, X_{t-1}, A_{t-1}, X_t)$$

is a stochastic kernel on $\mathcal{A}$ given $\underbrace{\mathcal{X} \times \mathcal{A} \times \cdots \times \mathcal{X} \times \mathcal{A} \times \mathcal{X}}_{2t-1 \text{ elements}}$

satisfying

$$\pi_t(\mathcal{A} | X_1, A_1, X_2, A_2, \ldots, X_{t-1}, A_{t-1}, X_t) = 1$$

for every $(X_1, A_1, X_2, A_2, \ldots, X_{t-1}, A_{t-1}, X_t)$.

Policy

Definition

If π_t is parametrized only by X_t, that is

$$\pi_t(\cdot|X_1, A_1, X_2, A_2, \ldots, X_{t-1}, A_{t-1}, X_t) = \pi_t(\cdot|X_t),$$

$\bar{\pi}$ is a Markov policy.
If for each t and $(X_1, A_1, X_2, A_2, \ldots, X_{t-1}, A_{t-1}, X_t)$, the policy π_t assigns mass one to a single point in $\mathcal{A}$, $\bar{\pi}$ is called a *deterministic (nonrandomized)* policy; if it assigns a distribution over $\mathcal{A}$, it is called *stochastic* or *randomized* policy.
If $\bar{\pi}$ is a Markov policy in the form of $\bar{\pi} = (\pi, \pi, \ldots)$, it is called a stationary policy.

A policy $\pi(\cdot|x)$ is a stationary Markov policy. We often work with such policies. If it is also deterministic, we denote it by $\pi(x)$.

Policy-Induced Transition Kernels

An agent is "following" a Markov stationary policy π whenever A_t is selected according to the policy $\pi(\cdot|X_t)$, i.e., $A_t = \pi(X_t)$ (deterministic) or $A_t \sim \pi(\cdot|X_t)$ (stochastic).
The policy π induces two transition probability kernels $\mathcal{P}^\pi : \mathcal{X} \to \mathcal{M}(\mathcal{X})$ and $\mathcal{P}^\pi : \mathcal{X} \times \mathcal{A} \to \mathcal{M}(\mathcal{X} \times \mathcal{A})$. For a (measurable) subset A of $\mathcal{X}$ and a (measurable) subset B of $\mathcal{X} \times \mathcal{A}$ and a deterministic policy π, denote

$$(\mathcal{P}^\pi)(A|x) \triangleq \int_{\mathcal{X}} \mathcal{P}(\mathrm{d}y|x, \pi(x)) \mathbb{I}_{\{y \in A\}},$$

$$(\mathcal{P}^\pi)(B|x, a) \triangleq \int_{\mathcal{X}} \mathcal{P}(\mathrm{d}y|x, a) \mathbb{I}_{\{(y, \pi(y)) \in B\}}.$$

Policy-Induced Transition Kernels

When we have a countable state-action space, we sometimes use summation instead of integrals. For example,

$$(\mathcal{P}^{\pi})(A|x) \triangleq \sum_{y \in \mathcal{X}} \mathcal{P}(y|x, \pi(x)) \mathbb{I}_{\{y \in A\}} = \sum_{y \in A} \mathcal{P}(y|x, \pi(x)).$$

So for a particular $y \in \mathcal{X}$, we have $(\mathcal{P}^{\pi})(y|x) = \mathcal{P}(y|x, \pi(x))$. Also we can extend the definition of $\mathcal{P}^{\pi}$ to following a policy for m-steps ($m \geq 1$) inductively. We use $(\mathcal{P}^{\pi})^m$ to denote such a transition kernel.

From Immediate to Long-Term Reward

RL problem: How to act so that some notion of long-term performance is maximized.
Q: What does long-term mean? How to quantify it?

Immediate Reward Problem

- At each round of interaction with its environment
  - An agent starts at a random state $X_1 \sim \rho \in \mathcal{M}(\mathcal{X})$
  - It chooses action $A_1 = \pi(X_1)$ (deterministic policy), and receives a reward of $R_1 \sim \mathcal{R}(\cdot|X_1, A_1)$.

We call each of these rounds an episode. Here the episode only lasts one time-step.

Q: How should this agent choose its policy in order to maximize its "performance"?

Q: What does performance mean?

Immediate Reward Problem

- At each round of interaction with its environment
  - An agent starts at a random state $X_1 \sim \rho \in \mathcal{M}(\mathcal{X})$
  - It chooses action $A_1 = \pi(X_1)$ (deterministic policy), and receives a reward of $R_1 \sim \mathcal{R}(\cdot|X_1, A_1)$.

We can talk about average (expected) reward that the agent receives within one episode as the measure of performance.
Average is over repeated interactions with the environment.
If we define the performance in this way, answering the question of how the agent should act to maximize this notion of performance is easy.

Immediate Reward Problem

Let us define expected reward as

$$r(x, a) \triangleq \mathbb{E}\left[R | X = x, A = a\right].$$

In order to maximize the expected reward, the best action depends on the state the agent initially starts with. At state x, it should choose

$$a^* \leftarrow \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, r(x, a).$$

This is the best, or *optimal*, action at state x
The optimal policy $\pi^* : \mathcal{X} \to \mathcal{A}$:

$$\pi^*(x) \leftarrow \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, r(x, a). \tag{1}$$

Optimal policy depends on the agent's initial state. It does not depend on initial distribution ρ.

Finite Horizon Task

The agent interacts with the environment for a fixed $T \geq 1$ number of steps.

- At each round (episode),
  - The agent starts at $X_1 \sim \rho \in \mathcal{M}(\mathcal{X})$.
  - It chooses action $A_1 = \pi(X_1)$ (or $A_1 \sim \pi(\cdot|X_1)$ for a stochastic policy)
  - The agent goes to the next-state $X_2 \sim \mathcal{P}(\cdot|X_1, A_1)$ and receives reward $R_1 \sim \mathcal{R}(\cdot|X_1, A_1)$.
  - (this process repeats for several steps until ...)
  - $X_T \sim \mathcal{P}(\cdot|X_{T-1}, A_{T-1})$.
  - $R_T \sim \mathcal{R}(\cdot|X_{T-1}, A_{T-1})$.

So we receive a reward sequence $(R_1, R_2, \ldots, R_T)$.

Q: How should we evaluate the performance of the agent as a function of the reward sequence?

Finite Horizon Task

A common choice for performance is to compute the sum of rewards:

$$G^\pi \triangleq R_1 + \ldots + R_T. \tag{2}$$

The r.v. G^π is called the return of following policy π.
Here the rewards received at all time steps are treated the same.
The agent just adds them together.

Finite Horizon Task

Another choice is to consider *discounted* sum of rewards. Given a discount factor $0 \leq \gamma \leq 1$, we define the return as

$$G^{\pi} \triangleq R_1 + \gamma R_2 + \ldots + \gamma^{T-1} R_T. \tag{3}$$

Whenever $\gamma < 1$, the reward that is received earlier contributes more to the return. Intuitively, this means that such a definition of return values earlier rewards more.

- A cookie today is better than a cookie tomorrow, and a cookie tomorrow is better than a cookie a week later.
- Financial interpretation (inflation rate).
- Marshmallow test (delayed gratification).

Smaller values of γ makes the agent more myopic.

Remark

The discount factor is a part of the problem definition.

From Return to Value Function

The return (3) (and (2) as a special case) is a random variable. To define a performance measure that is not random, we compute its expectation.

$$V^{\pi}(x) \triangleq \mathbb{E}\left[\sum_{t=1}^{T} \gamma^{t-1} R_t | X_1 = x\right].$$

This is the expected value of return if the agent starts at state x and follows policy π. The function $V^{\pi} : \mathcal{X} \to \mathbb{R}$ is called the value function of π.

From Return to Value Function

Let us look at the case of $T = 1$ a bit closer. The value function V^π at state x is

$$V^\pi(x) = \mathbb{E}\left[R_1 | X = x\right].$$

This is similar to $r(x, a) = \mathbb{E}\left[R | X = x, A = a\right]$ with the difference that $r(x, a)$ is conditioned on both x and a, whereas V^π is conditioned on x.

In V^π, the choice of action is determined by the policy π, i.e., at state x, $a = \pi(x)$ or $A \sim \pi(\cdot|x)$.

If we define

$$r^\pi(x) \triangleq \mathbb{E}\left[R | X = x\right]$$

with $A \sim \pi(\cdot|x)$, we get that $r^\pi = V^\pi$.

For $T > 1$, V^π captures the long-term (discounted) average of the rewards, instead of the expected immediate reward captures by r^π.

How to Get the Optimal Policy?

Let us focus on $T = 1$ again.
Recall from before that for the immediate reward maximization problem, the optimal policy was

$$\pi^*(x) \leftarrow \underset{a \in \mathcal{A}}{\operatorname{argmax}} \, \mathbb{E}\left[R | X = x, A = a\right].$$

Getting the optimal policy from V^π "seems" less straightforward.

How to Get the Optimal Policy?

We need to search over the space of all deterministic or stochastic policies. If we denote the space of all stochastic policies by

$$\Pi = \{ \pi \, : \, \pi(\cdot|x) \in \mathcal{M}(\mathcal{A}), \forall x \in \mathcal{X} \}$$

we need to find

$$\pi^* \leftarrow \underset{\pi \in \Pi}{\operatorname{argmax}} \, V^\pi.$$

It turns out that this problem is not too difficult when $T = 1$. As the values of V^π at two different states $x_1, x_2 \in \mathcal{X}$ do not have any interaction with each other, we find the optimal policy at each state separately.

How to Get the Optimal Policy?

For each $x \in \mathcal{X}$,

$$V^{\pi}(x) = \int \mathcal{R}(\mathrm{d}r|x,a)\pi(\mathrm{d}a|x) = \int \pi(\mathrm{d}a|x)r(x,a).$$

Find a $\pi(\cdot|x)$ that maximizes $V^{\pi}(x)$ means that

$$\sup_{\pi(\cdot|x)\in\mathcal{M}(\mathcal{A})} \int \pi(\mathrm{d}a|x)r(x,a).$$

The maximizing distribution can concentrate all its mass at the action a^* that maximizes $r(x,a)$ (assuming it exists). Therefore, $\pi^*(a|x) = \delta(a - \mathrm{argmax}_{a'\in\mathcal{A}}\, r(x,a'))$ (or equivalently, $\pi^*(x) = \mathrm{argmax}_{a'\in\mathcal{A}}\, r(x,a')$) is an optimal policy at state x. This is for $T = 1$. For $T > 1$, the problem would be more complicated.

Episodic Tasks

In some scenarios, there is a final time T that the episode ends (or terminates), but it is not fixed a priori.

- Chess
- Finding a goal within a maze
- Robot successfully picks an object

The episode terminates whenever the agent reaches a certain state x_{terminal} within the state space, i.e., it terminates whenever $X_T = x_{\text{terminal}}$. The length of the episode T is a random variable.

Episodic Tasks

The definition of the return and value functions is as before:
For $0 \leq \gamma \leq 1$, we have

$$G^{\pi} \triangleq \sum_{k=1}^{T} \gamma^{k-1} R_k,$$

and

$$V^{\pi}(x) \triangleq \mathbb{E}\left[G^{\pi} | X_1 = x\right].$$

Remark

If $\gamma < 1$, these definitions are always well-defined. If $\gamma = 1$, we need to ensure that the termination time T is finite. Otherwise, the summation might be divergent (just imagine that all R_t are equal to 1).

Continuing Tasks

Sometimes the interaction between the agent and its environment does not break into episodes that terminates. It goes on continually forever.

- Life-long robot
- Chemical plant that is supposed to work for a long time
- An approximate model for finite-horizon problem with very large T.

Continuing Tasks

Consider the sequence of rewards $(R_1, R_2, \dots)$ generated after the agent starts at state $X_1 = x$ and follows policy π. Given the discount factor $0 \leq \gamma < 1$, the return is

$$G_t^\pi \triangleq \sum_{k \geq t} \gamma^{k-t} R_k. \tag{4}$$

Continuing Tasks

Definition (Value Functions)

The (state-)value function V^π and the action-value function Q^π for a policy π are defined as follows: Let $(R_t; t \geq 1)$ be the sequence of rewards when the process is started from a state X_1 (or (X_1, A_1) for the action-value function) drawn from a positive probability distribution over $\mathcal{X}$ (or $\mathcal{X} \times \mathcal{A}$) and follows the policy π for $t \geq 1$ (or $t \geq 2$ for the action-value function). Then,

$$V^\pi(x) \triangleq \mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^{t-1} R_t | X_1 = x\right],$$

$$Q^\pi(x, a) \triangleq \mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^{t-1} R_t | X_1 = x, A_1 = a\right].$$

Continuing Tasks

- The value function V^π evaluated at state x is the expected discounted return of following the policy π from state x.
- The action-value function Q^π evaluated at (x, a) is the expected discounted return when the agent starts at state x, takes action a, and then follows policy π.

Remark

If $\gamma = 0$, $Q^\pi = \mathbb{E}[R_1 | X_1 = x, A_1 = a]$. This is the same as the expected immediate reward $r(x, a)$. The same way that we could easily compute the optimal action using $r(x, a)$ in the finite-horizon problem with $T = 1$, we can use Q^π (in continual task) in order to easily compute the optimal policy.

Optimal Policy and Optimal Value Function

Q: What does it mean for an agent to act optimally?
First, let us think about how we can compare two (Markov stationary) policies π and π'. We say that π is better than or equal to π' (i.e., $\pi \geq \pi'$) iff $V^{\pi}(x) \geq V^{\pi'}(x)$ for all states.
Optimal policy: If we can find a policy π^* that satisfies $\pi^* \geq \pi$ for any π, we call it an optimal policy.

Remark

There may be more than one optimal policy, but their values are

Optimal Policy and Optimal Value Function

If we denote Π as the space of all stationary Markov polices, this means that

$$\pi^* \leftarrow \underset{\pi \in \Pi}{\operatorname{argmax}} V^{\pi},$$

where one of the maximizers is selected in an arbitrary way. The value function of this policy is the called the optimal value function, and is denoted by V^{π^*} or simply V^*.
We can also define the optimal policy based on Q^{π}, i.e.,

$$\pi^* \leftarrow \underset{\pi \in \Pi}{\operatorname{argmax}} Q^{\pi}.$$

The optimal action-value function is denoted by Q^{π^*} or Q^*.

Optimal Policy and Optimal Value Function

For the immediate reward maximization problem (finite horizon with $T = 1$), it is easy to find the optimal value function. Recall that $\pi^*(x) \leftarrow \operatorname{argmax}_{a \in \mathcal{A}} r(x, a)$ (1), so for any $x \in \mathcal{X}$,

$$V^*(x) = V^{\pi^*}(x) = \max_{a \in \mathcal{A}} r(x, a).$$

It is clear that for any $\pi : \mathcal{X} \to \mathcal{A}$,

$$V^*(x) = \max_{a \in \mathcal{A}} r(x, a) \geq r(x, \pi(x)).$$

The conclusion would be the same for stochastic policies.

Optimal Policy and Optimal Value Function

When we go to continual tasks (or even finite horizon with $T > 1$), we can ask several questions:

- Does any optimal policy exist? Maybe no single policy can dominate all others for all states.
  For example, it is imaginable that at best we can only hope to find a π^* that is better than any other policy π only on a proper subset of $\mathcal{X}$.
- Is the optimal policy necessarily a stationary policy?
  Isn't it possible to have a policy $\bar{\pi} = \{\pi_1, \pi_2, \ldots\}$ that depends on the time step and acts better than any stationary policy $\bar{\pi} = \{\pi, \pi, \ldots\}$?
- More pragmatic question: How can we find an optimal policy (if it exists)?

Optimal Policy and Optimal Value Function

When we go to continual tasks (or even finite horizon with $T > 1$), we can ask several questions:

- (Planning Problem) How can we find an optimal policy (if it exists) given the model $\mathcal{P}$ and $\mathcal{R}$?
- (RL Problem) How we can *learn* π^* (or a close approximation) without actually knowing the MDP, but only have samples coming from interacting with the MDP?

Q-Learning

It takes a while before we learn the necessary background before facing the first RL algorithms. Before that, let's have a sneak peak at one of the most well-known RL algorithms: Q-Learning.
Q-Learning is the quintessential RL algorithm, introduced by Christopher Watkins [Watkins, 1989, Chapter 7 – Primitive Learning]. Q-Learning itself is an example of the Temporal Difference (TD) learning [Sutton, 1988].

Q-Learning

Require: Step size $\alpha \in (0, 1]$

Initialize $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ arbitrary, except that for x_{terminal}, set $Q(x_{\text{terminal}}, \cdot) = 0$.

for each episode **do**

  Initialize $X_1 \sim \rho$

  for each step t of episode **do**

    $A_t \sim \pi(\cdot|X_t)$,

    Take action A_t, observe X_{t+1} and R_t

    Update $Q(X_t, A_t)$ using the following update rule

$$Q(X_t, A_t) \leftarrow Q(X_t, A_t) + \alpha \left[R_t + \gamma \max_{a' \in \mathcal{A}} Q(X_{t+1}, a') - Q(X_t, A_t) \right].$$

  end for

end for

Q-Learning

Variety of policies can be selected. A commonly-used one is ε-greedy policy:

$$A_t = \begin{cases} \text{argmax}_{a \in \mathcal{A}} Q(X_t, a) & \text{w.p. } 1 - \varepsilon \\ \text{uniform}(\mathcal{A}) & \text{w.p. } \varepsilon \end{cases}$$

Usually the value of ε is small and may go to zero as the agent learns more about its environment.

Remark

Under certain conditions, including how the learning rate α should be selected, the Q-Learning algorithm on finite state-action MDPs can be guaranteed to converge to the optimal action-value function Q^*.

GENETIC ALGORITHM (GA)

Answering the AI Question

- How do we get an agent to act intelligently?
- One answer:
  - Humans have evolved a brain
    - Which enables us to act intelligently
- Idea:
  - Mimic Darwinian evolution to evolve intelligent agents
- Intelligence is a specific instance of evolution
  - Evolving a species to survive in a niche
  - Via adaptation of species through survival of the fittest
- More general idea:
  - Evolve programs to solve problems

Evolutionary Approaches to AI

- John Holland 1975
  - "Adaptation in Natural and Artificial Systems"
- Developed the idea of a genetic algorithm
  - Searching via sampling hyperplane partitions of the search space
  - Still just search over a space
- Broader sense:
  - Somehow representing solutions to a problem
  - And allowing solutions to be combined into new ones
    - And testing whether the new ones are improvements

Evolutionary Computation

- Computational procedures patterned after biological evolution.
- Search procedure that probabilistically applies search operators to set of points in the search space
- Evolutionary Algorithm, Evolutionary Programming, Genetic Algorithm & Genetic Programming.

Evolution …

Lamarck and others:

- Species "transmute" over time

Darwin and Wallace:

- Consistent, heritable variation among individuals in population
- Natural selection of the fittest

Mendel and genetics:

- A mechanism for inheriting traits
- genotype → phenotype mapping

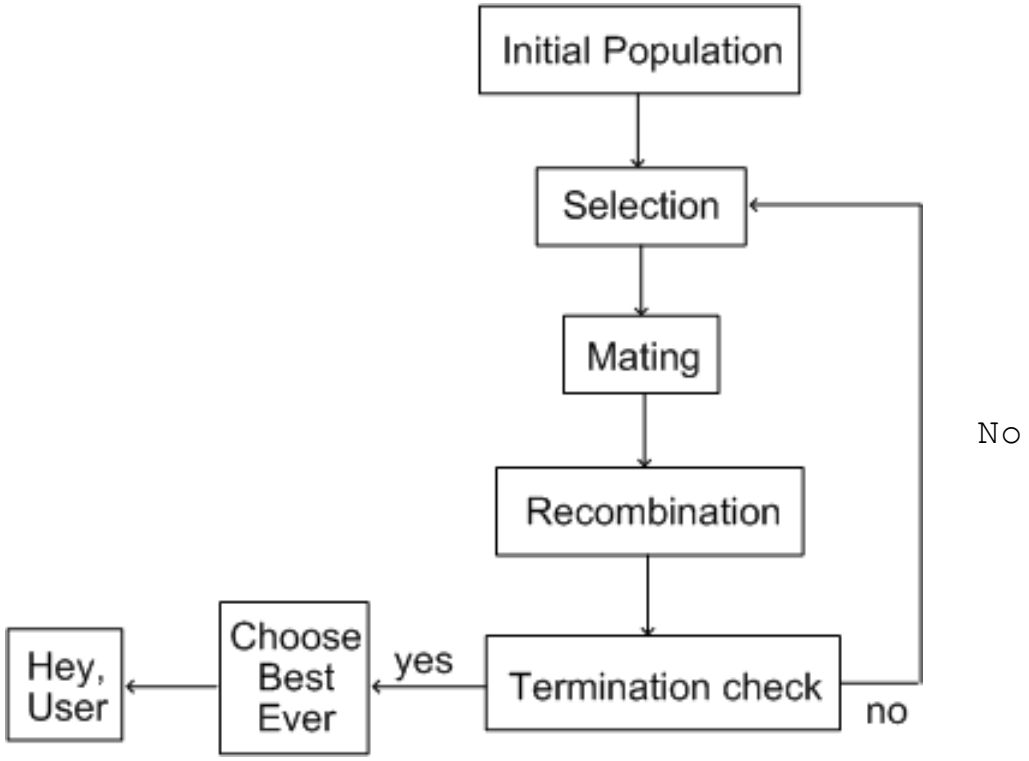
Genetic Algorithms (GAs) and Genetic Programming (GP)

- Genetic Algorithms
  - Optimising parameters for problem solving
  - Fix a format for the parameters in a problem solution
  - Represent the parameters in the solution(s)
    - As a “bit” string normally, but often something else
  - Evolve answers in this representation
- Genetic Programming
  - Representation of solutions is richer in general
  - Solutions can be interpreted as programs
  - Evolutionary process is very similar
  - Covered in next lecture

Overview of Classical GAs

- Representation of parameters is a bit string
  - Solutions to a problem represented in binary
  - 10101001001110101010101
    - Everything in computers is represented in binary
- Start with a population (fairly large set)
  - Of possible solutions known as **individuals**
- Combine possible solutions by swapping material
  - Choose the “best” solutions to swap material between and kill off the worse solutions
  - This generates a new set of possible solutions
- Requires a notion of “fitness” of the individual
  - Base on an evaluation function with respect to the problem
  - Dependent on properties of the solution

The Canonical Genetic Algorithm



No

GA($Fitness, Fitness\_threshold, p, r, m$)

- *Initialize:* $P \leftarrow p$ random hypotheses
- *Evaluate:* for each h in P, compute $Fitness(h)$
- While $[\max_h Fitness(h)] < Fitness\_threshold$
  1. *Select:* Probabilistically select $(1 - r)p$ members of P to add to P_S.

$$\Pr(h_i) = \frac{Fitness(h_i)}{\Sigma_{j=1}^{p} Fitness(h_j)}$$

  2. *Crossover:* Probabilistically select $\frac{r \cdot p}{2}$ pairs of hypotheses from P. For each pair, $\langle h_1, h_2 \rangle$, produce two offspring by applying the Crossover operator. Add all offspring to P_s.
  3. *Mutate:* Invert a randomly selected bit in $m \cdot p$ random members of P_s
  4. *Update:* $P \leftarrow P_s$
  5. *Evaluate:* for each h in P, compute $Fitness(h)$

- Return the hypothesis from P that has the highest fitness.

The Initial Population

- Represent solutions to problems
  - As a bit string of length L
  - (Difficult part of using GA's – see later)
- Choose an initial population size
  - Generate length L strings of 1s & 0s randomly
  - Sometimes (rarely) done more intelligently
- Strings are sometimes called chromosomes
  - Letters in the string are called "genes"
    - See later for the (bad) analogies
  - We call the bit-string "individuals"

Represent

$$(Outlook = Overcast \vee Rain) \wedge (Wind = Strong)$$

by

$Outlook$	$Wind$
011	10

Represent

IF $Wind = Strong$ THEN $PlayTennis = yes$

by

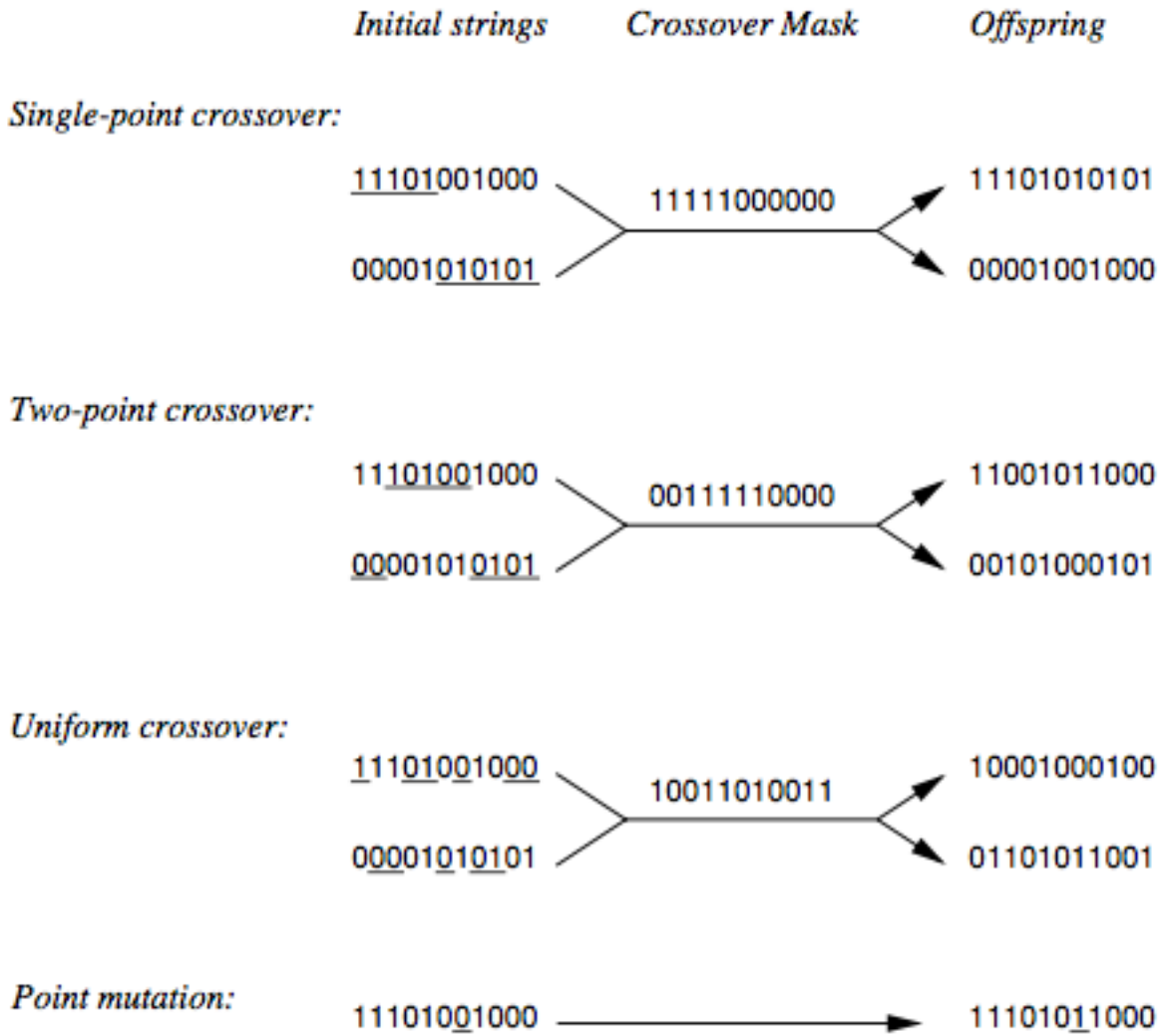
$Outlook$	$Wind$	$PlayTennis$
111	10	10

The Selection Process

- We are going to combine pairs of members of the current population to produce offspring
  - So, we need to choose which pairs to combine

- Will use an **evaluation function, g(c)**
  - Which is dependent on desirable properties of the solutions (defining this is often tricky)

- Use g(c) to define a **fitness function**


Initial strings
Crossover Mask
Offspring
Single-point crossover:
11101001000
00001010101
11111000000
11101010101
00001001000
Two-point crossover:
11101001000
00001010101
00111110000
11001011000
00101000101
Uniform crossover:
11101001000
00001010101
10011010011
10001000100
01101011001
Point mutation:
11101001000
11101011000

Calculating Fitness

- Use a **fitness function**
  - To assign a probability of each individual being used in the mating process for the next generation
- Usually use fitness(c) = g(c)/A
  - Where A = average of g(c) over the entire population
- Other constructions are possible

Using the Fitness Function

- Use fitness to produce an intermediate population IP
  - From which mating will occur
- Integer part of the fitness value is:
  - The number of copies of individuals guaranteed to go in IP
- Fractional part is:
  - A probability that an extra copy will go into IP
  - Use a probabilistic function (roll dice) to determine whether
    - The extra copies are added to the IP

Examples of Selection

- Example #1
  - Suppose the average of g over the population is 17
  - And for a particular individual c_0, $g(c_0) = 25$
  - Fitness(c_0) = 25/17 = 1.47
  - Then 1 copy of c_0 is guaranteed to go into IP
  - And the probability of another copy going in is 0.47
- Example #2
  - Suppose $g(c_1) = 14$
  - No copy of c_1 is guaranteed to go into IP
  - The probability of one copy going in is 14/17 = 0.82

The Mating Process

- Pairs from the Intermediate Population
  - Are chosen randomly
  - Could be the same individual twice
- They are combined with a probability p_c
  - Their offspring are produced by recombination
    - See later slides
  - Offspring go into the next generation population
- The process is repeated until
  - The next generation contains the required number of individuals for the next population
  - Often this is taken as the current population size
    - Which keeps the population size constant (not a bad idea)

Analogy with Natural Evolution

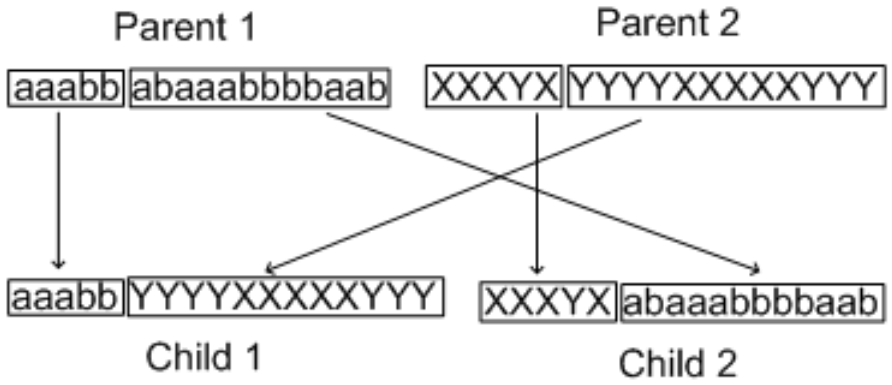
- Analogy holds:
  - Some very fit individuals may be unlucky, and:
    (a) not find a partner
    (b) not be able to reproduce with the partner they find
  - Some less fit individuals may be lucky and:
    - Find a partner and produce hundreds of offspring!
- Analogy breaks:
  - There is no notion of sexes
    - Although there are some types of asexual worms, etc., ….
  - Individuals can mate with themselves
    - Although cells reproduce alone…

The Recombination Process

- The population will only evolve to be better
  - If best parts of the best individuals are combined
- Crossover techniques are used
  - Which take material from each individual
  - And adds it to “offspring” individuals
- Mutation techniques are used
  - Which randomly alter the offspring
  - Good for getting out of local maxima

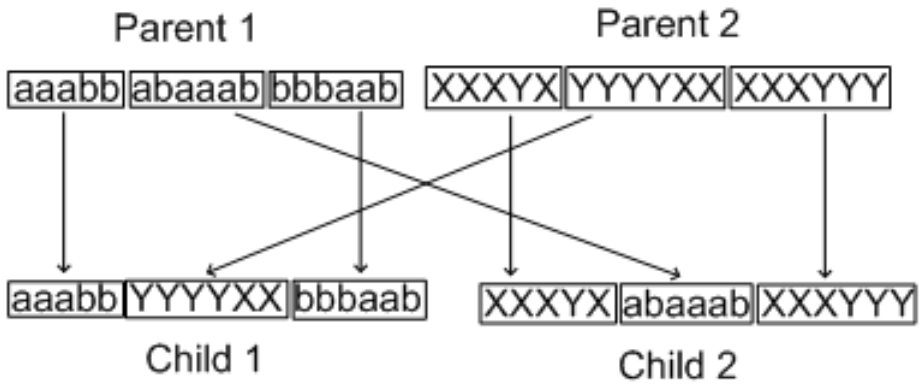
One-point crossover

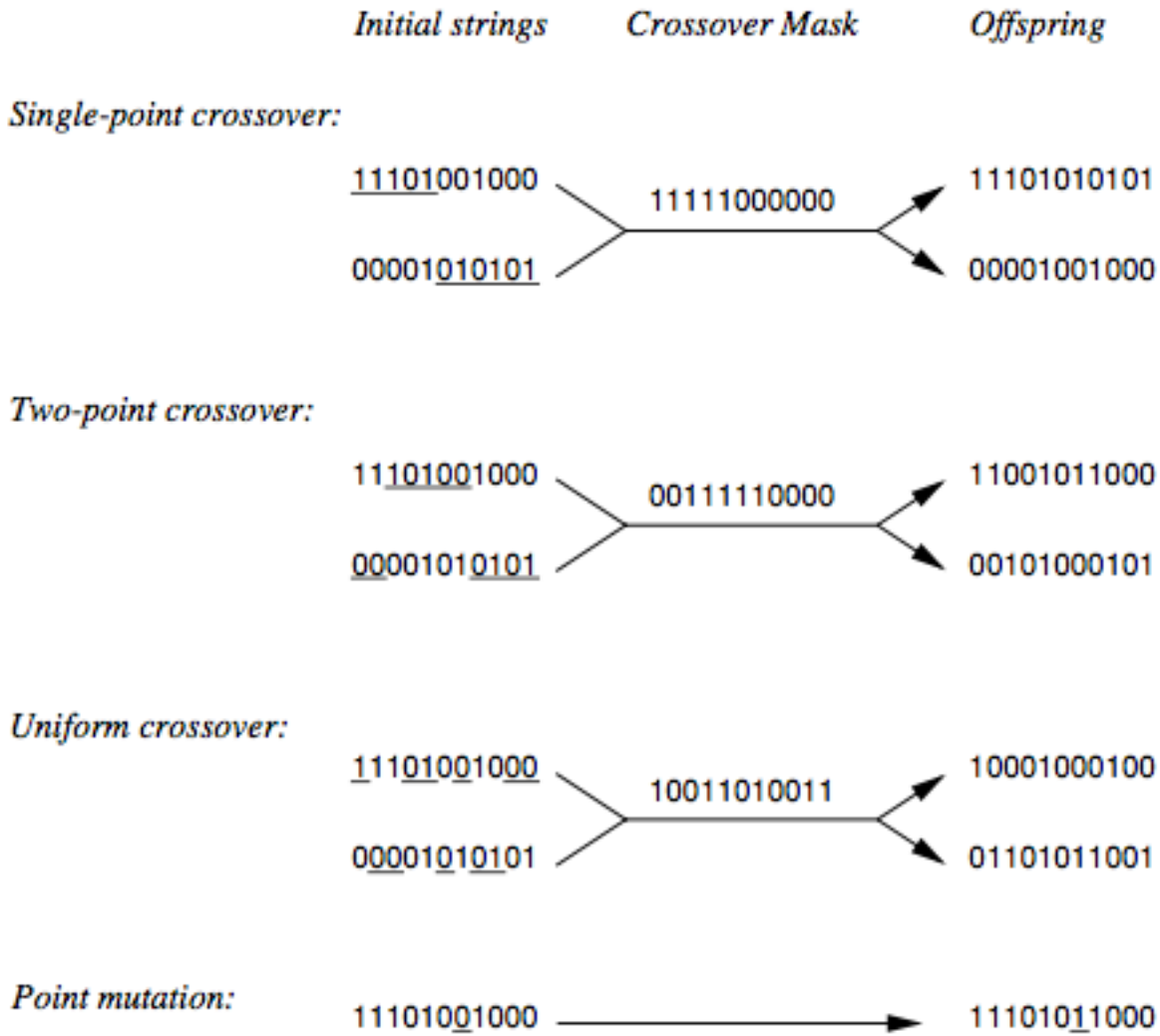
- Randomly choose a single point in both individuals
  - Both have the same point
  - Split into LHS_1+RHS_1 and LHS_2+RHS_2
- Generate two offspring from the combinations
  - Offspring 1: LHS_1+RHS_2
  - Offspring 2: LSH_2+RHS
- Example: (X,Y,a,b are all ones and zeros)



Two-point Crossover

- Two points are chosen in the strings
- The material falling between the two points
  - is swapped in the string for the two offspring
- Example:

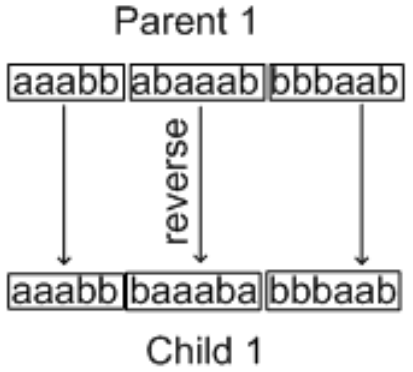



Initial strings
Crossover Mask
Offspring
Single-point crossover:
11101001000
00001010101
11111000000
11101010101
00001001000
Two-point crossover:
11101001000
00001010101
00111110000
11001011000
00101000101
Uniform crossover:
11101001000
00001010101
10011010011
10001000100
01101011001
Point mutation:
11101001000
11101011000

Inversion

- Another possible operator to mix things up
  - Takes the material from only on parent
- Randomly chooses two points
  - Takes the segment in-between the points
  - Reverses it in the offspring

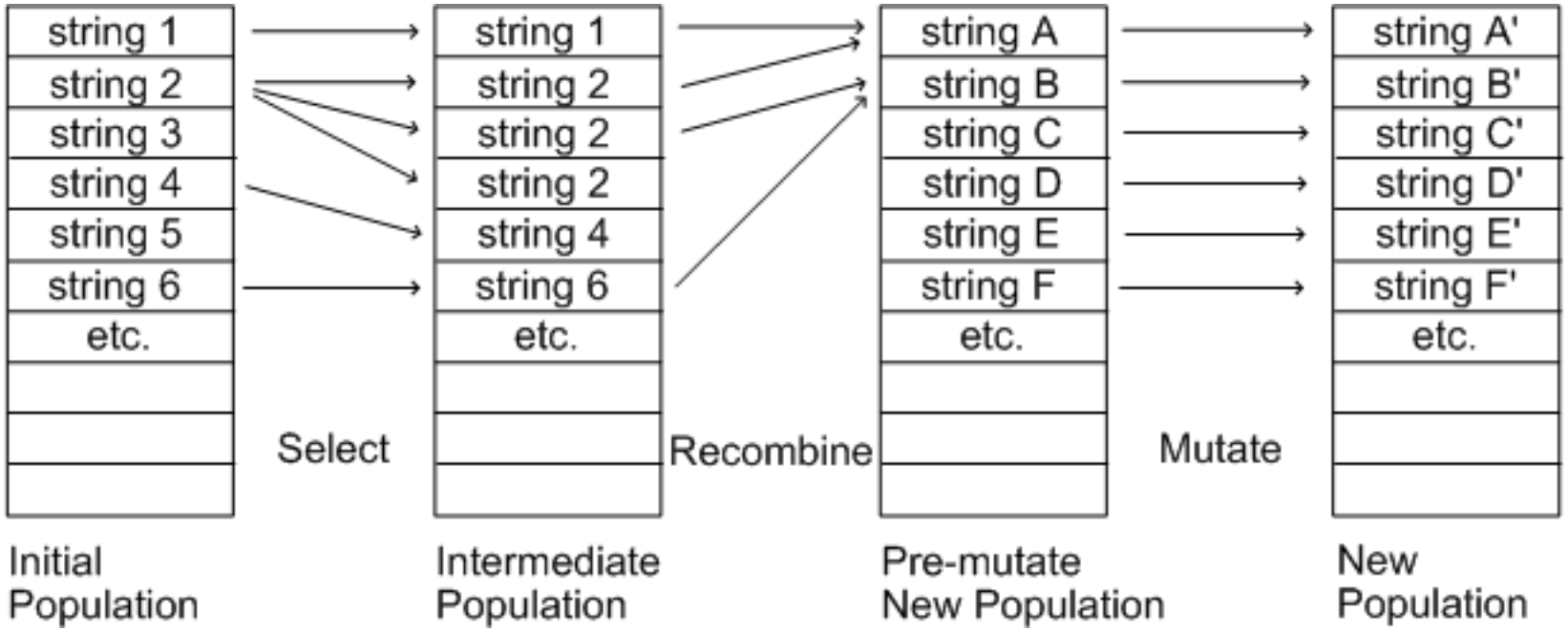
Example:



The Mutation Process

- Recombination keeps large strings the same
  - Not just randomly jumbling up the bit-strings (important point)
  - Produces a large range of possible solutions
  - But might get into a local maxima
- Random mutation
  - Each bit in every offspring produced
    - Is “flipped” from 0 to 1 or vice versa
    - With a given probability (usually pretty small < 1%)
  - May help avoid local maxima
    - In natural evolution mutation is almost always deleterious
- Could use probability distribution
  - To protect the best individuals in population
  - But is this good for avoiding local maxima?

Schematic Overview for Producing the Next Generation



Selection Process

Fitness proportionate selection:

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^{p} Fitness(h_j)}$$

... can lead to *crowding*

Tournament selection:

- Pick h_1, h_2 at random with uniform prob.
- With probability p, select the more fit.

Rank selection:

- Sort all hypotheses by fitness
- Prob of selection is proportional to rank

The Termination Check

- Termination may involve testing whether an individual solves the problem to a good enough standard
  - Not necessarily having found a definitive answer
- Alternatively, termination may occur after a fixed time
  - Or after a fixed number of generations
- Note that the best individual in a population
  - May not be as good as the best in a previous popn
- So, your GA should:
  - Record the best from each generation
  - Output the best from all populations as the answer

First Big Problem: Representing Solutions

- Difficulty 1:
  - Encoding solutions as bit strings in the first place
  - ASCII strings require 8 bits (=1 byte) per letter
  - Solutions can become very long
    - So that evolution takes many cycles to converge
- Difficulty 2:
  - Avoiding redundancy
  - Many bit strings produced by recombination and mutation will not
  - represent possible solutions at all
    - How do we evaluate such possibilities (obviously bad???)
  - Situation is better when solutions are continuous
    - Over a set of real-valued numbers or integers
  - Situation is worse when there is a finite number of solutions

Second Big Problem: Evaluation Functions

- Fitness function is not to be confused
  - With the evaluation function
- Evaluation function must, if possible:
  - Return a real-valued number
  - Be quick to calculate (this will be done many, many times)
  - Distinguish between different individuals
- Problem when populations have evolved
  - All individuals score highly with respect to fitness
  - So we may have to do some fixing to spread them out
    - Otherwise they all have roughly equal chance of reproducing
      - And evolution will have stopped

An Example Application to Transportation System Design

- Taken from the ACM Student Magazine
  - Undergraduate project of Ricardo Hoar & Joanne Penner
- Vehicles on a road system
  - Modelled as individual agents using ANT technology
    - (Also an AI technique)
- Want to increase traffic flow
- Uses a GA approach to evolve solutions to:
  - The problem of timing traffic lights
  - Optimal solutions only known for very simple road systems
- Details not given about bit-string representation
  - But traffic light switching times are real-valued numbers over a continuous space

Transport Evaluation Function

- Traffic flow is increased if:
  - The average time a car is waiting is decreased
  - Alternatively, the total waiting time is decreased
- They used this evaluation function:

$$\text{wait} = \sum_{i \in Cars} \frac{w_i}{d_i + w_i}$$

Figure 3b: Fitness function for sequence evaluation where w_i denotes total waiting time and d_i denotes total driving time for car *i*.

- W_i is the total waiting time for car i and d_i is the total driving time for car i.

Results

- Reduction in waiting times for a simple road:

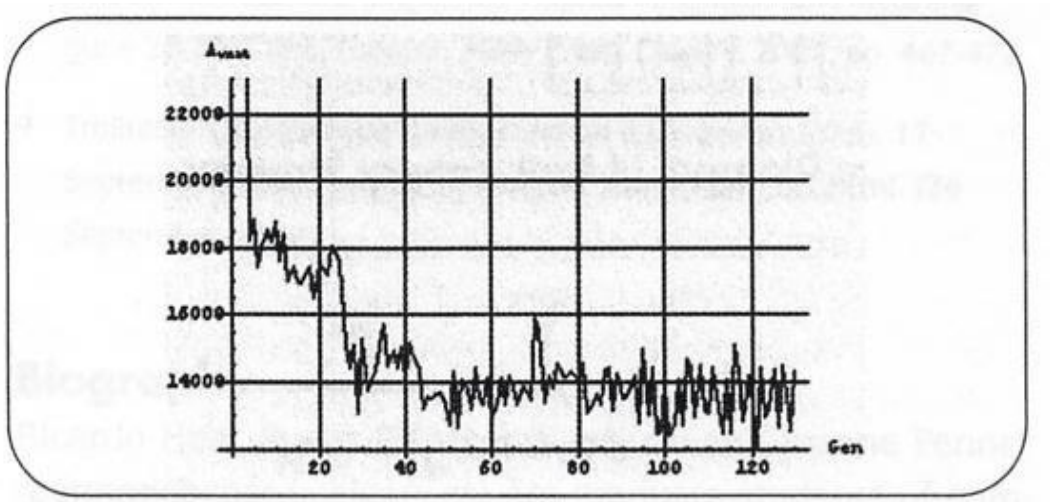


Figure 4b: Improvement in fitness over 130 generations.

- Solutions similar to (human) established solutions
  - Verification that the system is acting rationally

Results

- Reduction in waiting times for a more complicated road system (not too complicated, though)

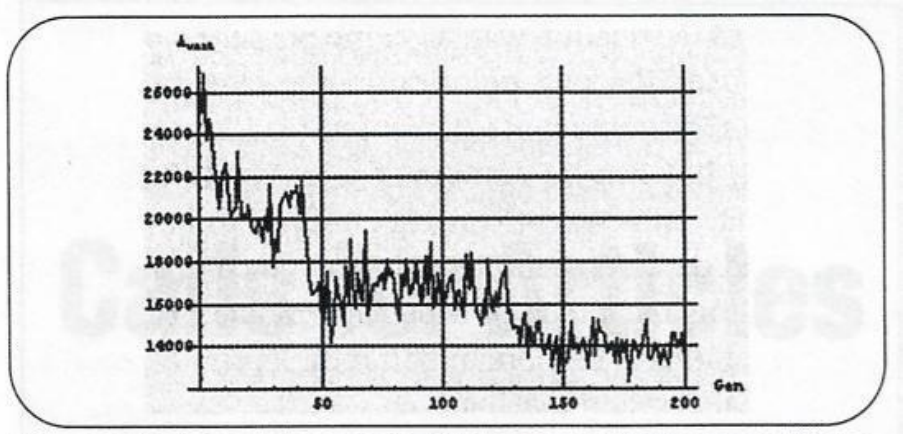
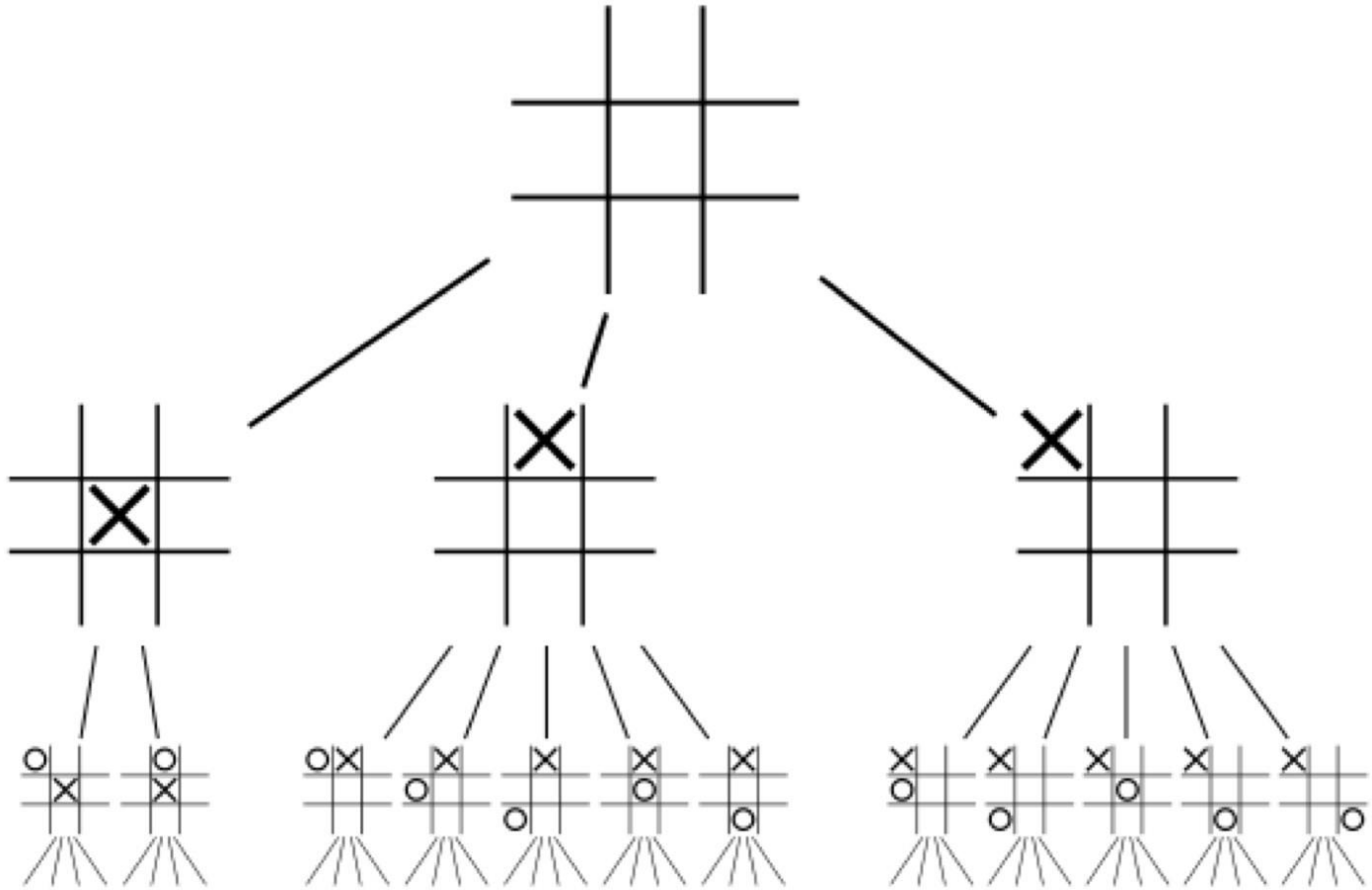


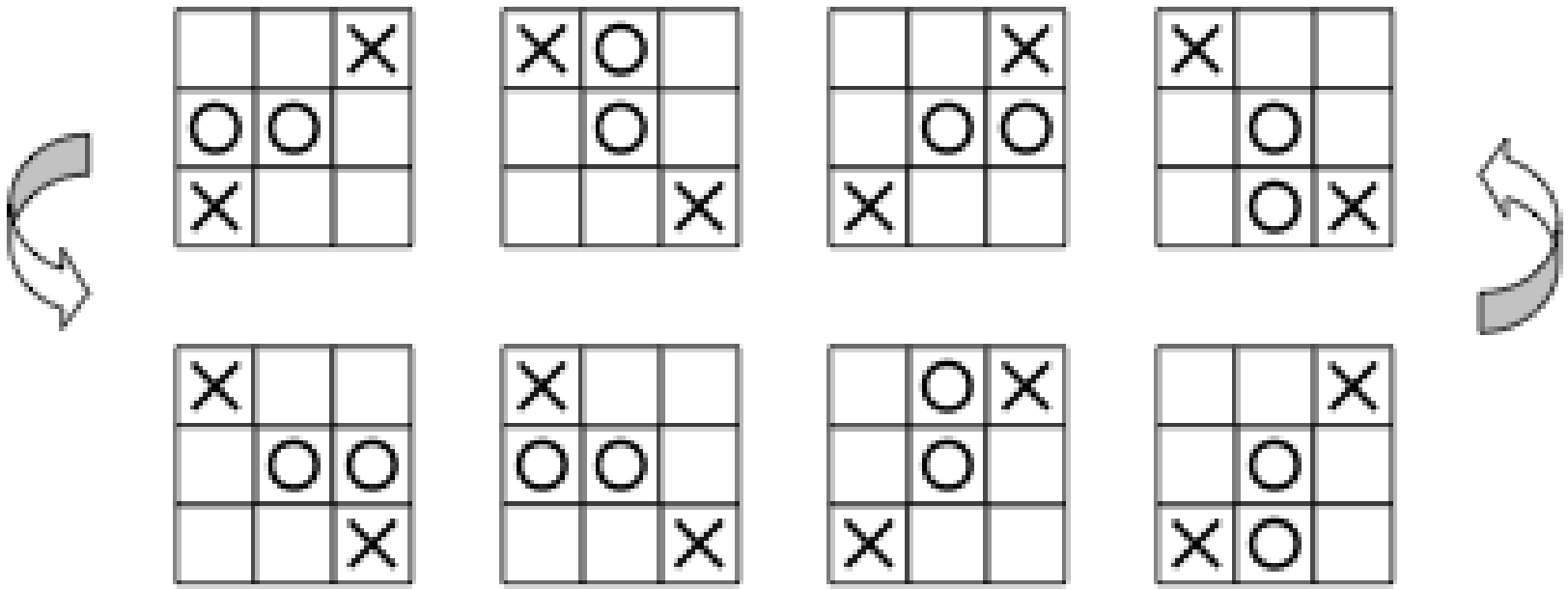
Figure 5b: Improvement in fitness over 200 generations.

- Roughly 50% decrease in waiting times
  - In both experiments

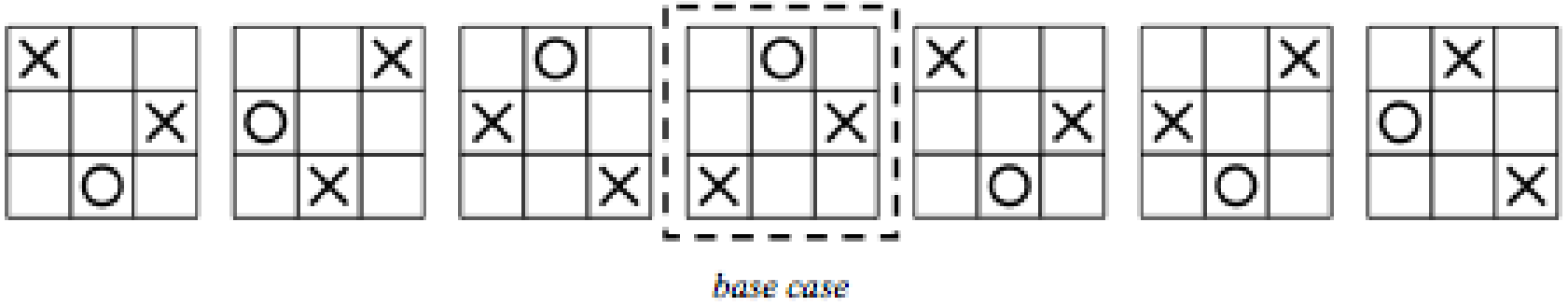
E.g. Learning tic-tac-toe using Genetic Algorithm

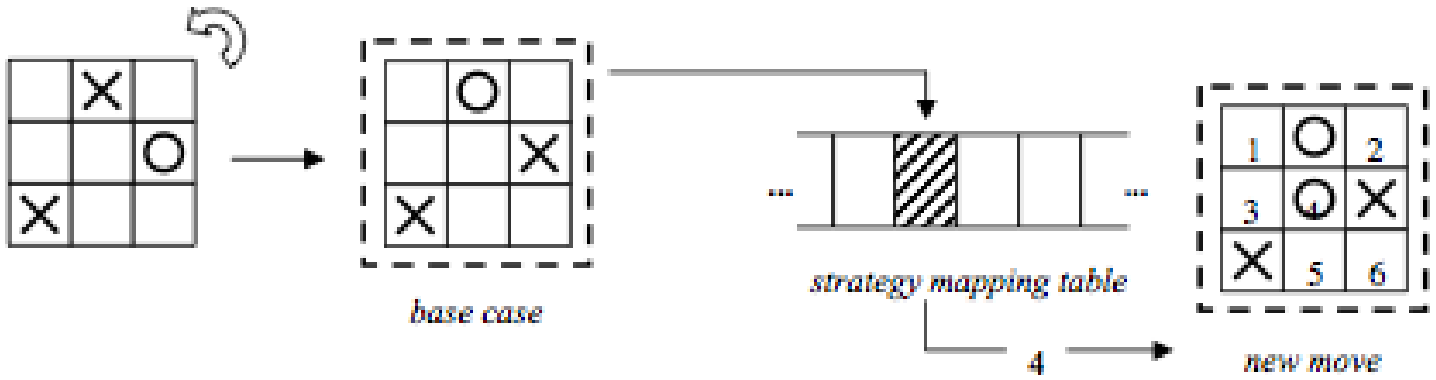


Search Space



Search Space





Using Genetic Algorithms

- Russell and Norvig:
  - If you can answer four questions:
    - What is the fitness [evaluation] function?
    - How is an individual represented?
    - How are individuals selected?
    - How do individuals reproduce?
  - Then why not try GAs for your problem?
    - Similar to neural networks and CSPs (might not be the best way to proceed, but it is quick and easy to get going)

THANK YOU